

ARM 프로세서 GPIO

* General Purpose Input Output (GPIO)

- 범용 입출력 포트

- 마이크로프로세서가 주변장치와 통신하기 위해 범용으로 사용되는 입력 출력 포트

Peripheral		STM32F103Cx		STM32F103Rx			STM32F103Vx	
Flash - Kbytes		32	64	32	64	128	64	128
SRAM - Kbytes		10	20	10	20		20	
Timers	General purpose	2	3	2	3		3	
	Advanced Control	1		1			1	
Communication	SPI	1	2	1	2		2	
	I ² C	1	2	1	2		2	
	USART	2	3	2	3		3	
	USB	1	1	1	1		1	
	CAN	1	1	1	1		1	
GPIOs		32		49			80	
12-bit synchronized ADC		2		2				
Number of channels		10 channels		16 channels				
CPU frequency		72 MHz						
Operating voltage		2.0 to 3.6 V						
Operating temperature		-40 to +85 °C / -40 to +105 °C						
Packages		LQFP48		LQFP64			LQFP100, BGA100	

ARM 프로세서 GPIO

* General Purpose Input Output (GPIO)

• 범용 입출력 포트

- 입출력을 마음대로 선택할 수 있고,
0과 1의 출력 신호를 임의로 만들어
제공할 수 있는 구조

Peripheral		STM32F103Cx		STM32F103Rx			STM32F103Vx	
Flash - Kbytes		32	64	32	64	128	64	128
SRAM - Kbytes		10	20	10	20		20	
Timers	General purpose	2	3	2	3		3	
	Advanced Control	1		1			1	
Communication	SPI	1	2	1	2		2	
	I ² C	1	2	1	2		2	
	USART	2	3	2	3		3	
	USB	1	1	1	1		1	
	CAN	1	1	1	1		1	
GPIOs		32		49			80	
12-bit synchronized ADC		2		2				
Number of channels		10 channels		16 channels				
CPU frequency		72 MHz						
Operating voltage		2.0 to 3.6 V						
Operating temperature		-40 to +85 °C / -40 to +105 °C						
Packages		LQFP48		LQFP64			LQFP100, BGA100	

ARM 프로세서 GPIO

* General Purpose Input Output (GPIO)

- 범용 입출력 포트
 - 입력으로 사용할 때에는 외부 인터럽트를 처리할 수 있도록 함
 - 입출력 방향 전환용 레지스터와 입출력용 데이터 레지스터 등 존재

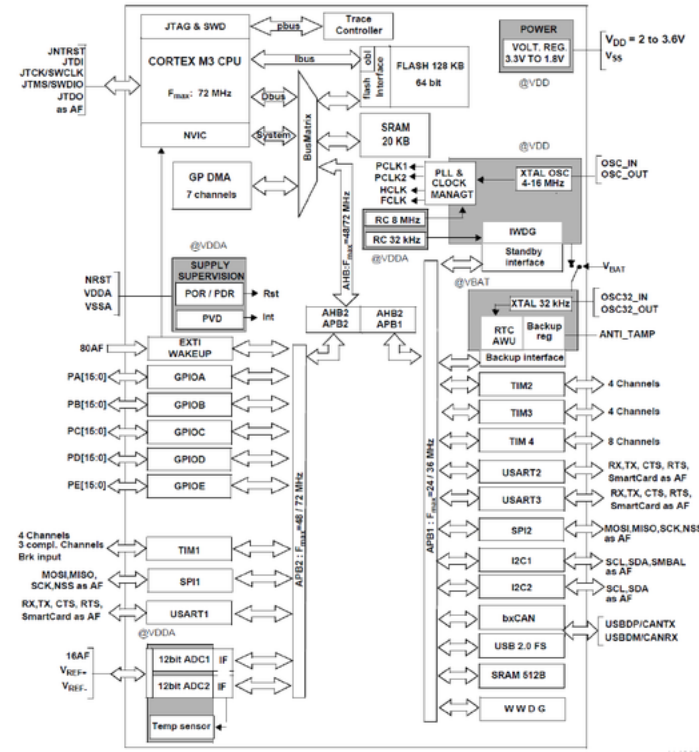
Peripheral	STM32F103Cx		STM32F103Rx		STM32F103Vx	
Flash - Kbytes	32	64	32	64	128	96
SRAM - Kbytes	10	20	10	20	20	20
Timers	General purpose	2	3	2	3	3
	Advanced Control	1		1		1
	SPI	1	2	1	2	2
Communication	I ² C	1	2	1	2	2
	USART	2	3	2	3	3
	USB	1	1	1	1	1
	CAN	1	1	1	1	1
	GPIOs	32		49		80
12-bit synchronized ADC	2		2		2	
Number of channels	10 channels		16 channels			
CPU frequency	72 MHz					
Operating voltage	2.0 to 3.6 V					
Operating temperature	-40 to +85 °C / -40 to +105 °C					
Packages	LQFP48		LQFP64		LQFP100	BGA100

ARM 프로세서 GPIO

* General Purpose Input Output (GPIO)

• 범용 입출력 포트

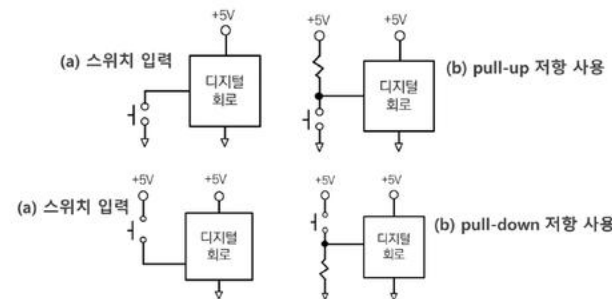
Peripheral		STM32F103Cx		STM32F103Rx			STM32F103Vx	
Flash - Kbytes		32	64	32	64	128	64	128
SRAM - Kbytes		10	20	10	20		20	
Timers	General purpose	2	3	2	3		3	
	Advanced Control	1		1			1	
Communication	SPI	1	2	1	2		2	
	I ² C	1	2	1	2		2	
	USART	2	3	2	3		3	
	USB	1	1	1	1		1	
	CAN	1	1	1	1		1	
GPIOs		32		49			80	
12-bit synchronized ADC		2		2				
Number of channels		10 channels		16 channels				
CPU frequency		72 MHz						
Operating voltage		2.0 to 3.6 V						
Operating temperature		-40 to +85 °C / -40 to +105 °C						
Packages		LQFP48		LQFP64			LQFP100, BGA100	



ARM 프로세서 GPIO

* General Purpose Input Output (GPIO)

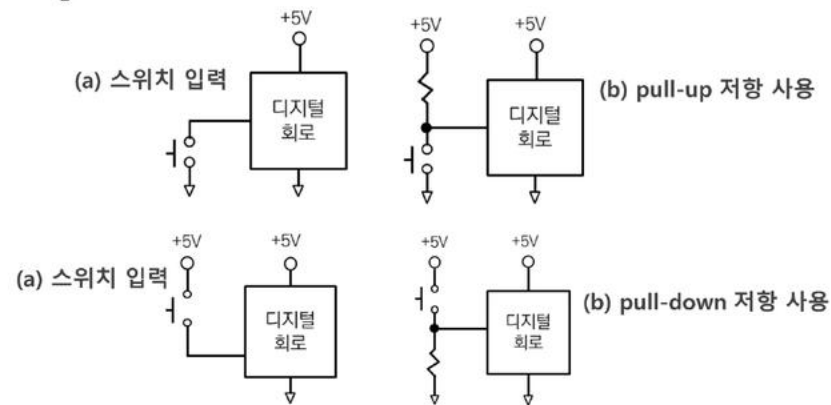
- 비트 레벨로 설정 및 클리어가 가능한 레지스터를 통해 한 포트 안에서는 다수의 비트를 하나의 명령으로 설정 및 클리어 가능
- Reset 후 기본 설정은 Input with Pull-Up



ARM 프로세서 GPIO

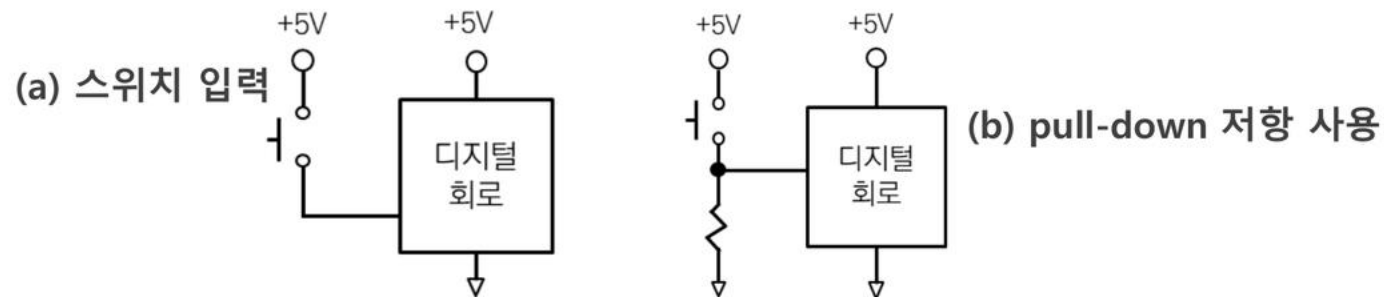
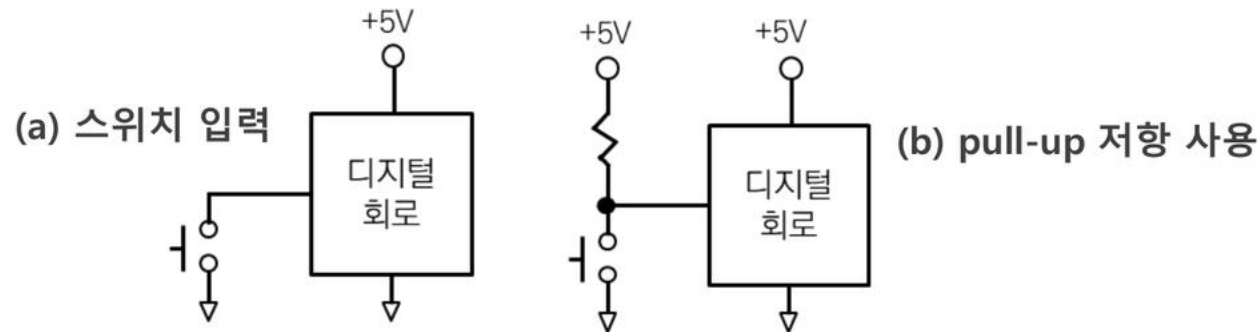
* Pinmode : setting for Pull-up or Pull-down

- pull-up 저항 : 논리적으로 High level 상태를 유지하기 위해 신호의 입력/출력 단자와 전원(Vcc) 단자 사이에 접속하는 저항



ARM 프로세서 GPIO

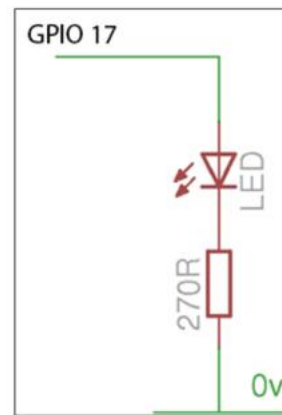
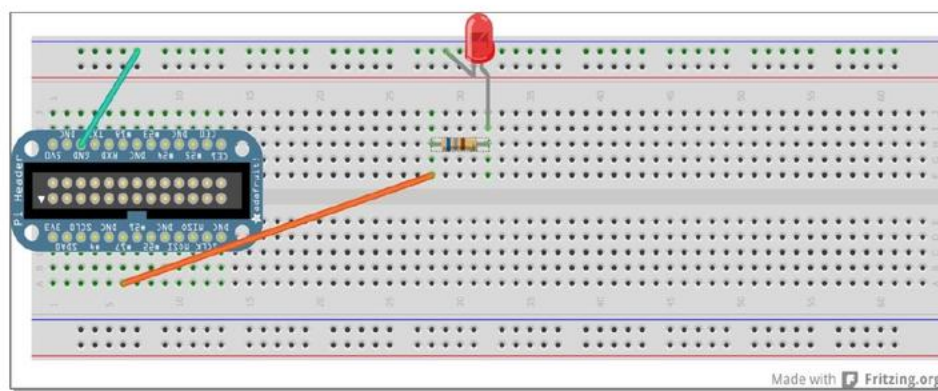
- * General Purpose Input Output (GPIO)
- * Pinmode : setting for Pull-up or Pull-down



ARM 프로세서 GPIO

* LED 연결방법

- 저항 220~330옴을 GPIO 16번, 18번에 연결
- GPIO의 Output 값을 High 로 설정해 주면 LED가 On, Low로 설정해 주면 Off가 되는 구성

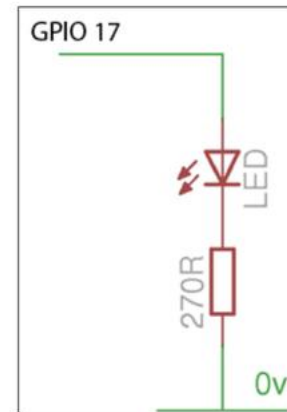
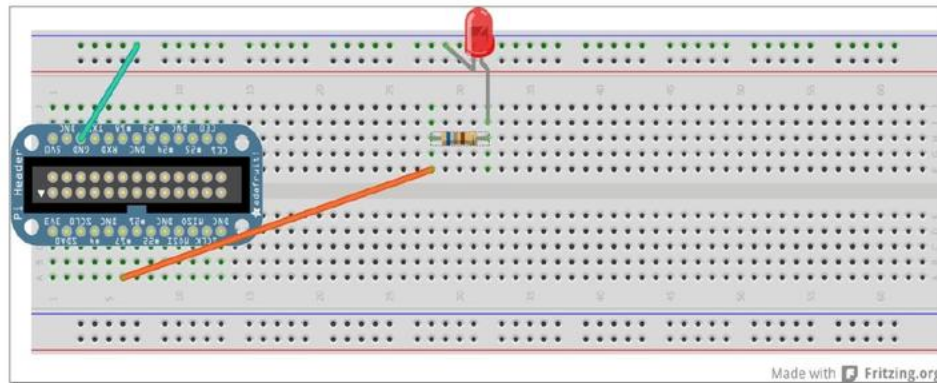


ARM 프로세서 GPIO

* 극성

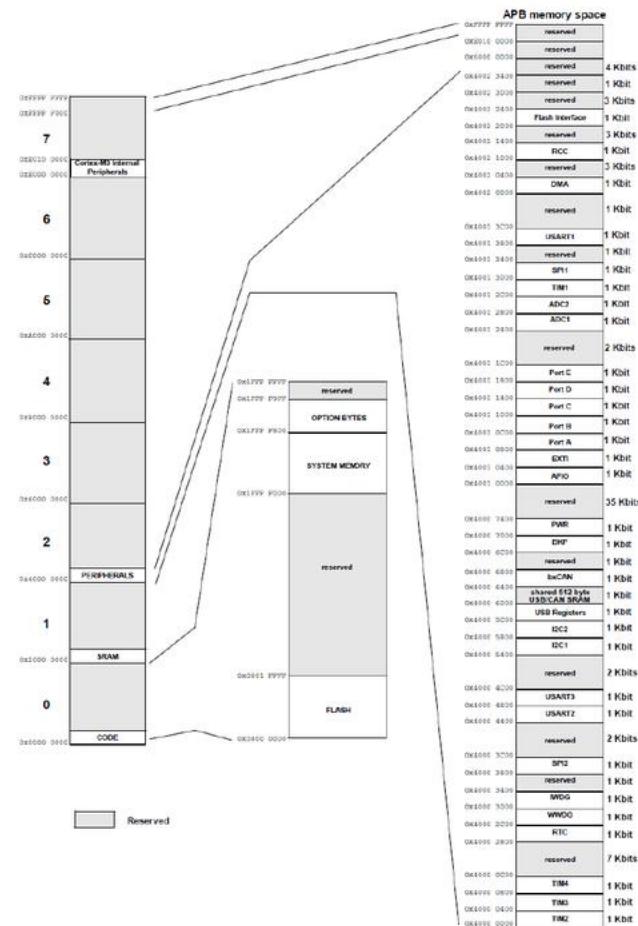
- 다리가 짧은 쪽이 Cathode -,
긴 쪽이 Anode +임

* 17번 포트 1번 bit 연결 사례



ARM 프로세서 GPIO

* Memory Map



ARM 프로세서 GPIO

• STM32 Cube HAL

- HAL드라이버는 풍부한 API 세트를 제공하고 애플리케이션 상위 계층과 쉽게 상호 작용하도록 설계됨
- 각 드라이버는 가장 일반적인 주변 장치 기능을 포함하는 함수 세트로 구성됨

ARM 프로세서 GPIO

- **STM32 Cube HAL**

- 각 드라이버의 개발은 드라이버 구조, 함수 및 매개변수 이름을 표준화하는 공통 API에 의해 주도함.
- HAL 드라이버는 드라이버 모듈 세트를 포함하며, 각 모듈은 독립형 주변 장치에 연결됨.

ARM 프로세서 GPIO

- HAL의 주요 기능

- 공통 주변 장치 기능을 포함하는
교차-가용성 API 세트와
특정 주변 장치 기능의 경우 확장 API
- 세 가지 API 프로그래밍 모델
: 폴링, 인터럽트, DMA

ARM 프로세서 GPIO

- HAL의 주요 기능
 - API는 RTOS를 준수
 - HAL API는 사용자 콜백 함수 메커니즘을 구현합니다.

ARM 프로세서 GPIO

- HAL의 주요 기능

- 주변 장치 초기화/초기화 해제
HAL API는 사용자 콜백 함수를 호출하여
주변 장치 시스템 수준의
초기화/초기화 해제
(클럭, GPIO, 인터럽트, DMA)를 수행 가능
- 주변 장치 인터럽트 이벤트

ARM 프로세서 GPIO

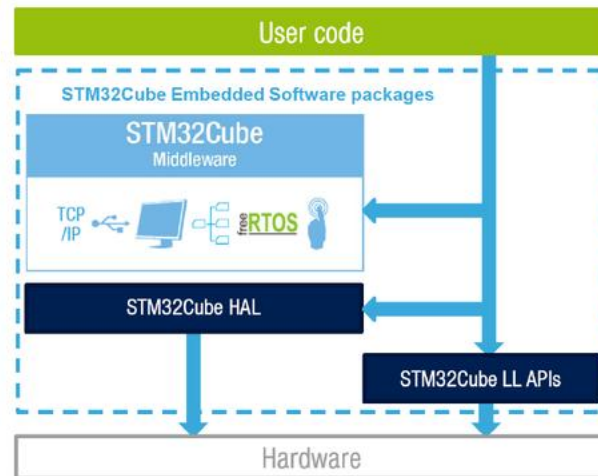
•GPIO 포트

•사용자 응용프로그램에서
GPIO를 접근할 수 있는 3가지 entry point

•1) Middleware stacks

•2) HAL APIs

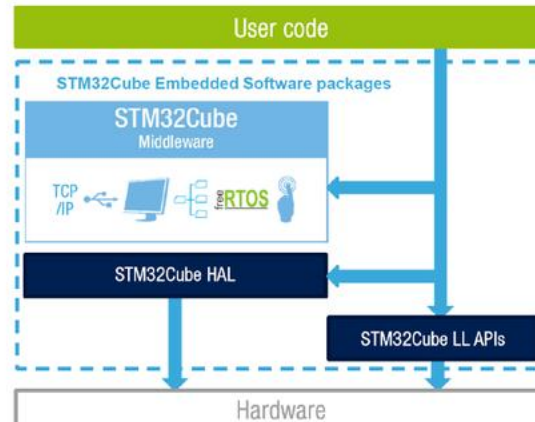
•3) LL APIs



ARM 프로세서 GPIO

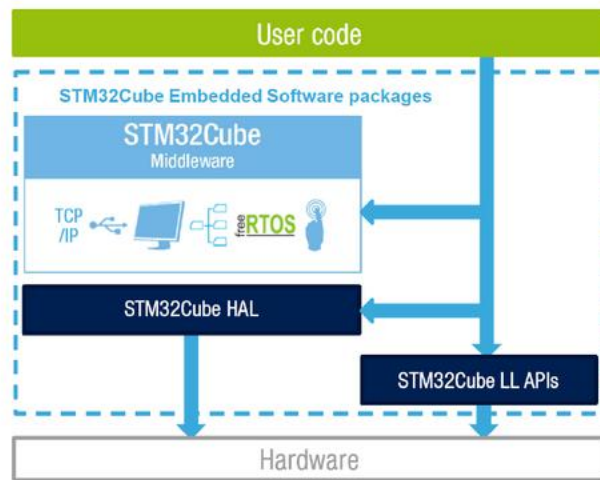
• 제약사항

- 동일한 주변장치 인스턴스에 대하여
LL과 HAL과는 동시에 사용할 수 없음.
(Sequentially 순차적으로
각각 달리 access하는 것은 허용됨)



ARM 프로세서 GPIO

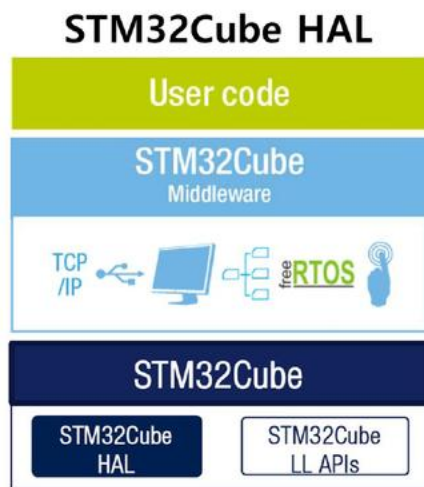
- 데이터시트에 나열된 각 I/O 포트의 특정 하드웨어 특성에 따라, GPIO 각 포트 비트는 소프트웨어적으로 여러 모드로 개별적으로 구성할 수 있음.



ARM 프로세서 GPIO

• 특징

- 고수준의 기능적 추상화 제공
- STM 계열 특정 시리즈 플랫폼에서 다른 시리즈 플랫폼으로 손쉽게 포팅 가능
- 다양한 주변 장치에 대한 높은 수준의 커버리지

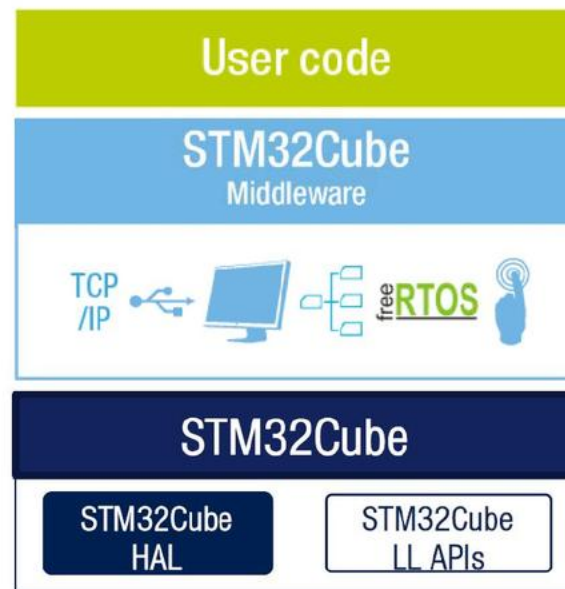


ARM 프로세서 GPIO

• 특징

- USB, TCP-IP, 그래픽, 터치 센스, RTOS와 같은 복잡한 미들웨어 통합
- STM32CubeMX와 함께 작동하여 초기화 코드 생성 가능

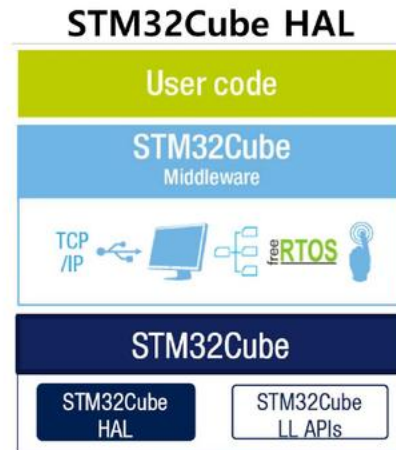
STM32Cube HAL



ARM 프로세서 GPIO

•제한 사항

- Low level 컨트롤을 희망하는 전문가 프로그래머에게는 활용가능한 범위에 있을 수 있음
- 높은 이식성은 더 큰 소프트웨어 용량 또는 적응 코드 실행 시간 증가를 초래할 수 있음

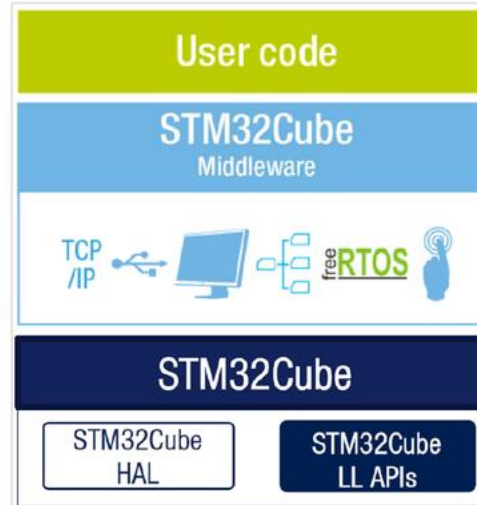


ARM 프로세서 GPIO

•특징

- Highly optimized, small code expression
- 레지스터 수준 액세스 가능
- 참조 매뉴얼을 closely follow

STM32Cube Low Level API

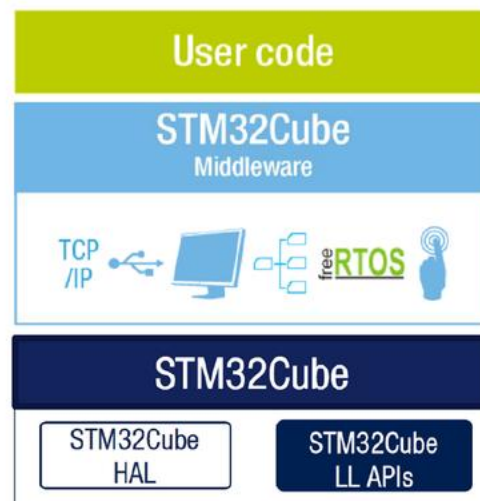


ARM 프로세서 GPIO

•특징

- 레지스터 수준에서의 디버깅 가능
- STM32CubeMX와 함께 작동하여
STM32L0/F0/F3/L4용
초기화 코드 생성 가능.
곧 더 많은 지원 예정

STM32Cube Low Level API

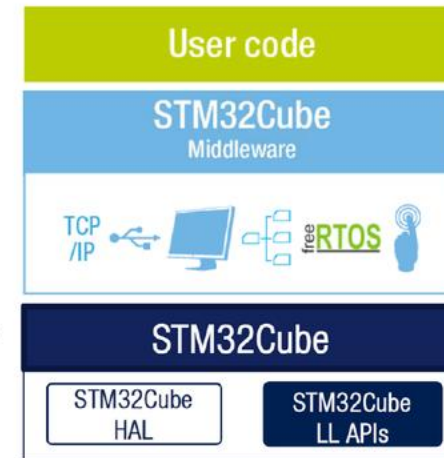


ARM 프로세서 GPIO

•제한 사항

- STM32 기기 전용으로,
시리즈 간에 직접 이식 불가
- USB와 같은 복잡한 주변장치와
일치하지 않음
- 런타임 추상화 부족으로,
개발자는 레지스터 레벨에서
주변장치 작동을 이해해야 함

STM32Cube Low Level API



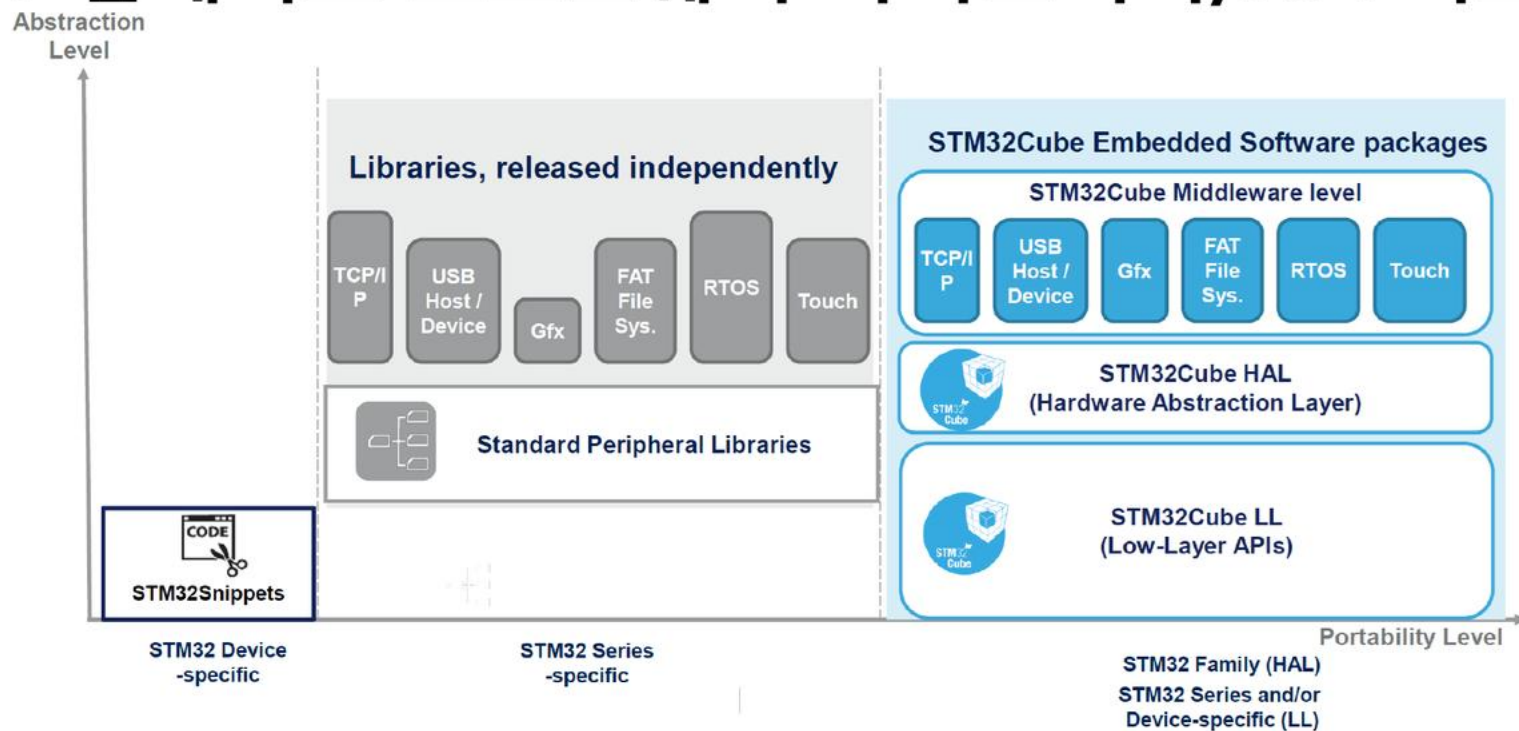
ARM 프로세서 GPIO

•ST 임베디드 소프트웨어 라이브러리/API 비교

	Portability	Optimization	Easy	Readiness	Hardware coverage
Standard Peripheral Library	++	++	+	++	+++
STM32 Cube HAL API	+++	+	++	+++	+++
STM32 Cube LL API	+	+++	+	++	++

ARM 프로세서 GPIO

• ST 임베디드 소프트웨어 라이브러리/API 비교



SECTION 03

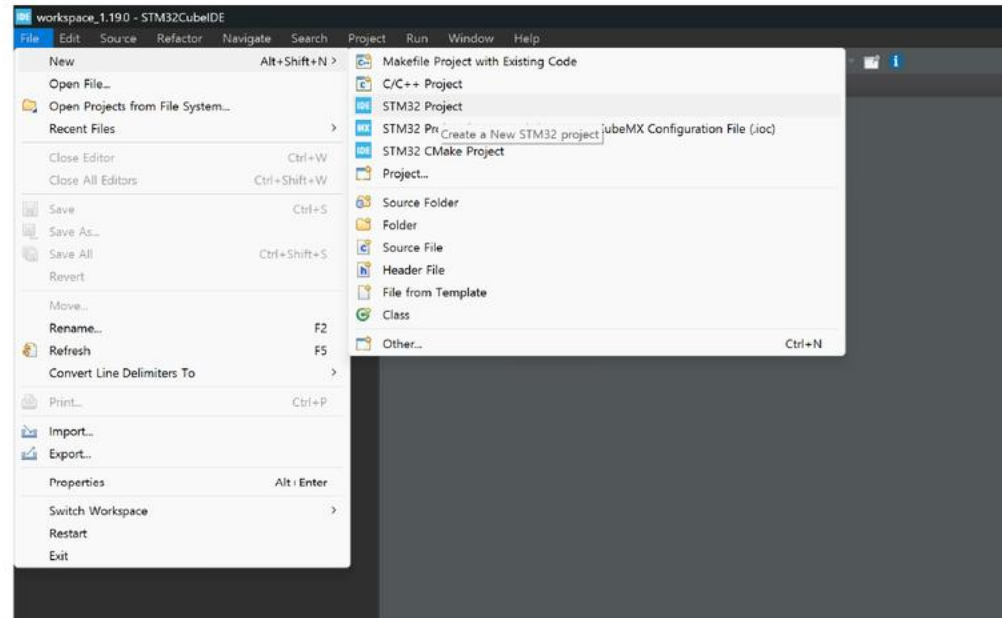
GPIO LED 제어 실습

STM32CubeIDE 프로젝트 생성부터 GPIO 설정, 코드 작성과 디버깅까지 단계별로 실습합니다.

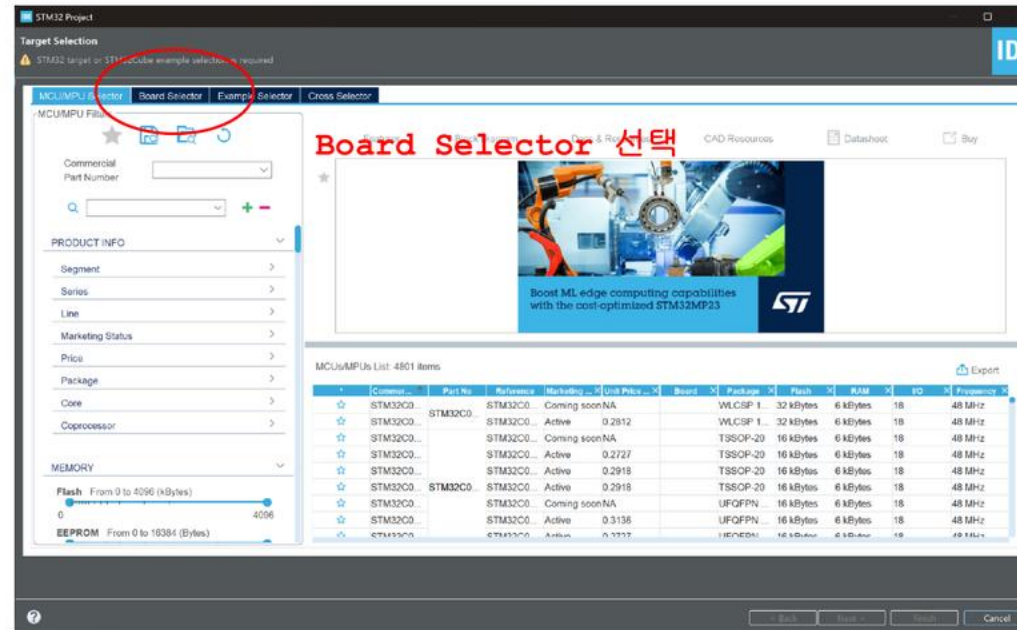


ARM GPIO를 통한 LED 제어 실습

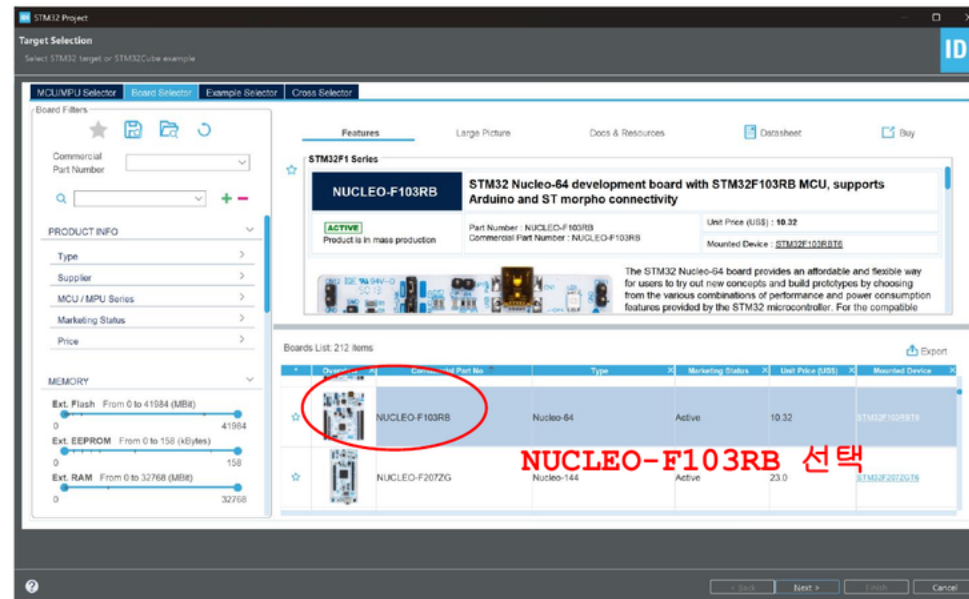
* STM32 Cube IDE에서 새로운 "STM32 Project" 프로젝트 생성



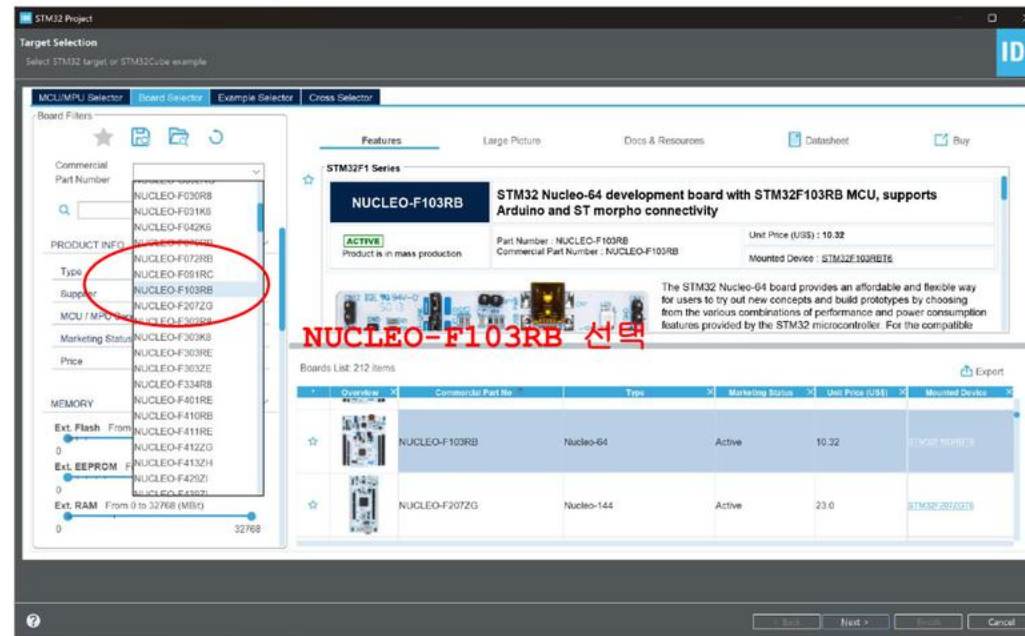
ARM GPIO를 통한 LED 제어 실습 임베디드 개발에서 각 프로젝트는 특정 보드와 연관되므로 보드 선택



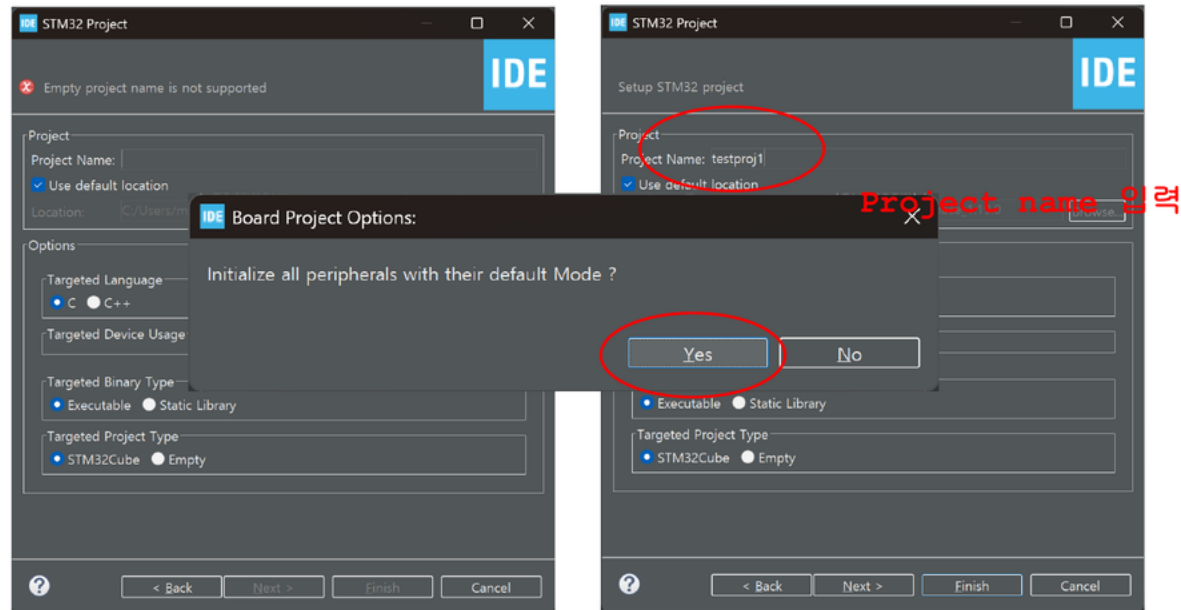
- ARM GPIO를 통한 LED 제어 실습
- 본 과정에서는 NUCLEO-F103RB 보드 사용



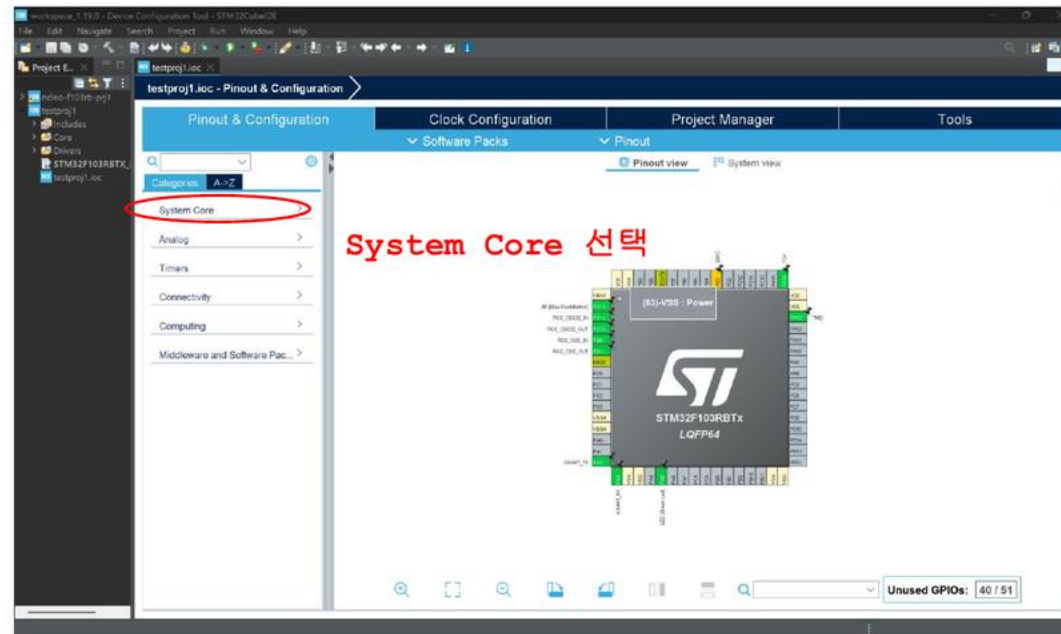
• ARM GPIO를 통한 LED 제어 실습



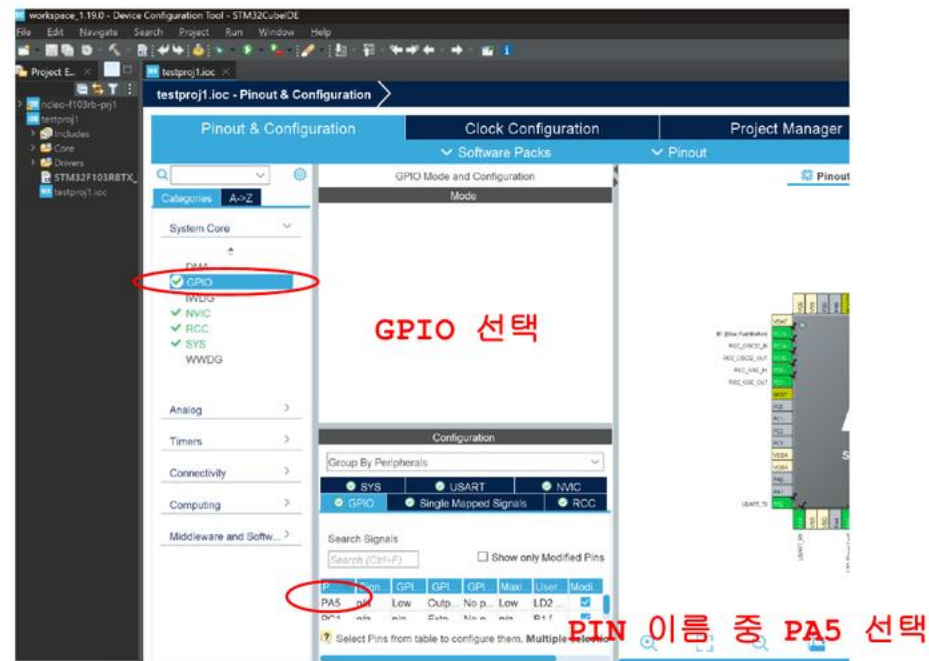
- ARM GPIO를 통한 LED 제어 실습
- Project Name 입력



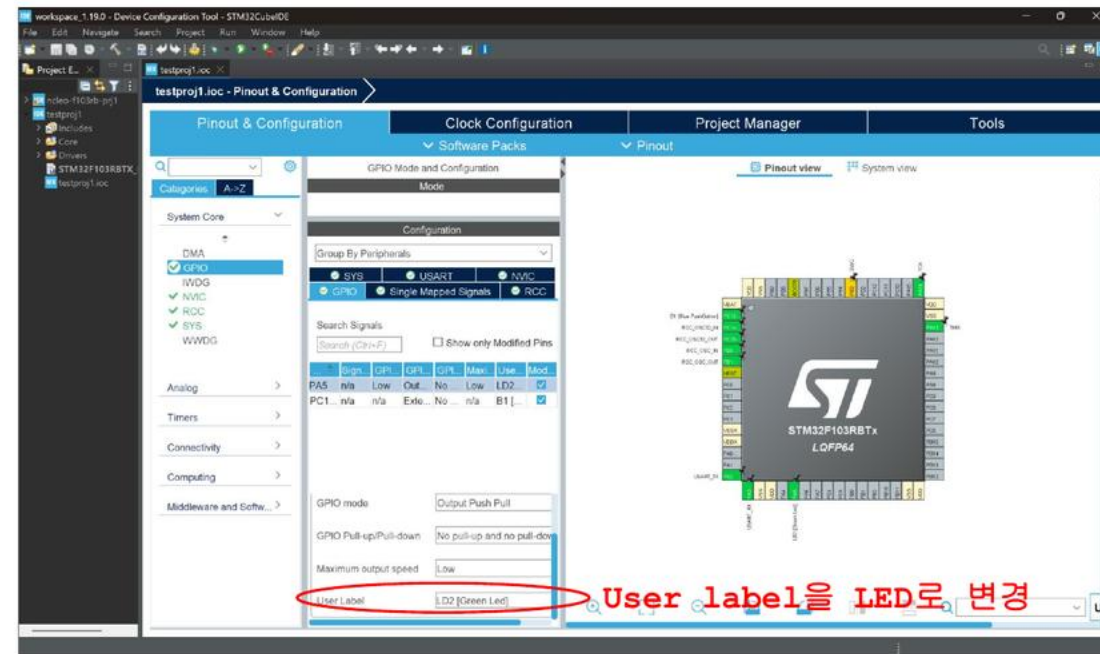
• ARM GPIO를 통한 LED 제어 실습



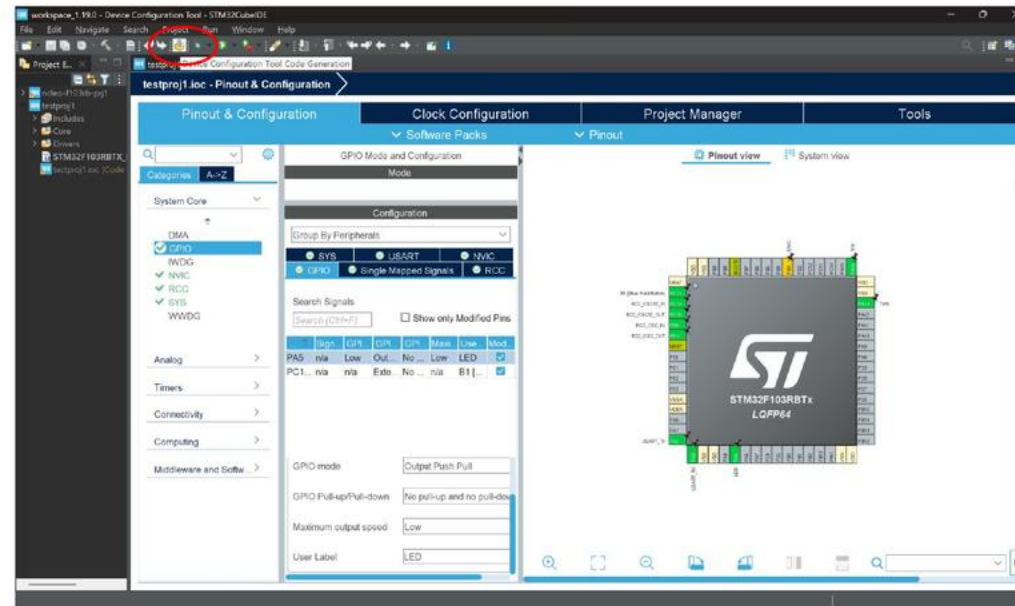
- ARM GPIO를 통한 LED 제어 실습
- System Core -> GPIO 선택
- Pin 목록에서 PA5 선택
- User Label을 LED로 변경



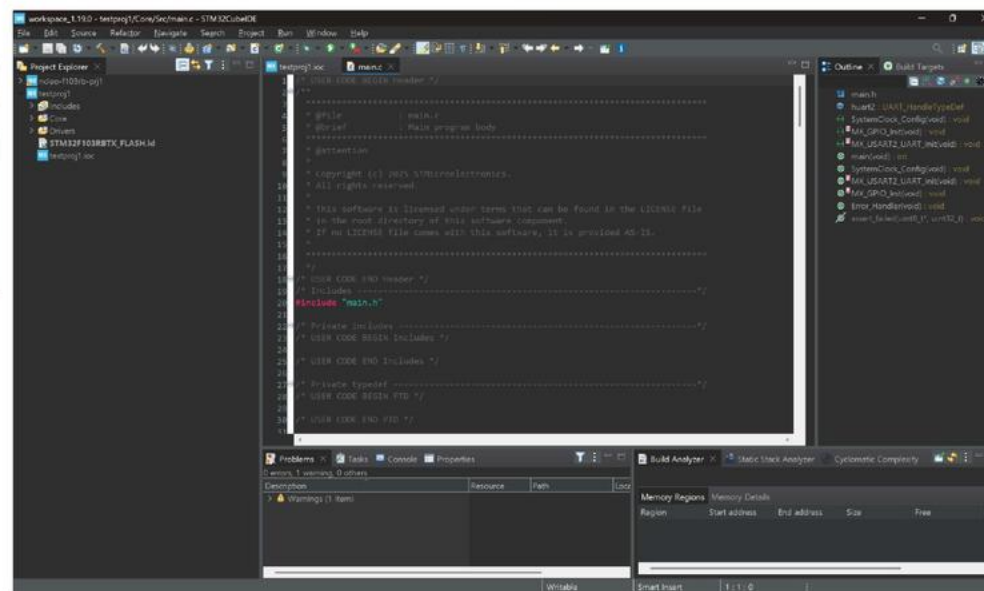
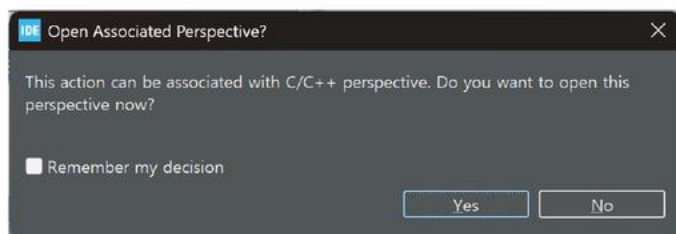
- ARM GPIO를 통한 LED 제어 실습
- User label 을 LED로 변경



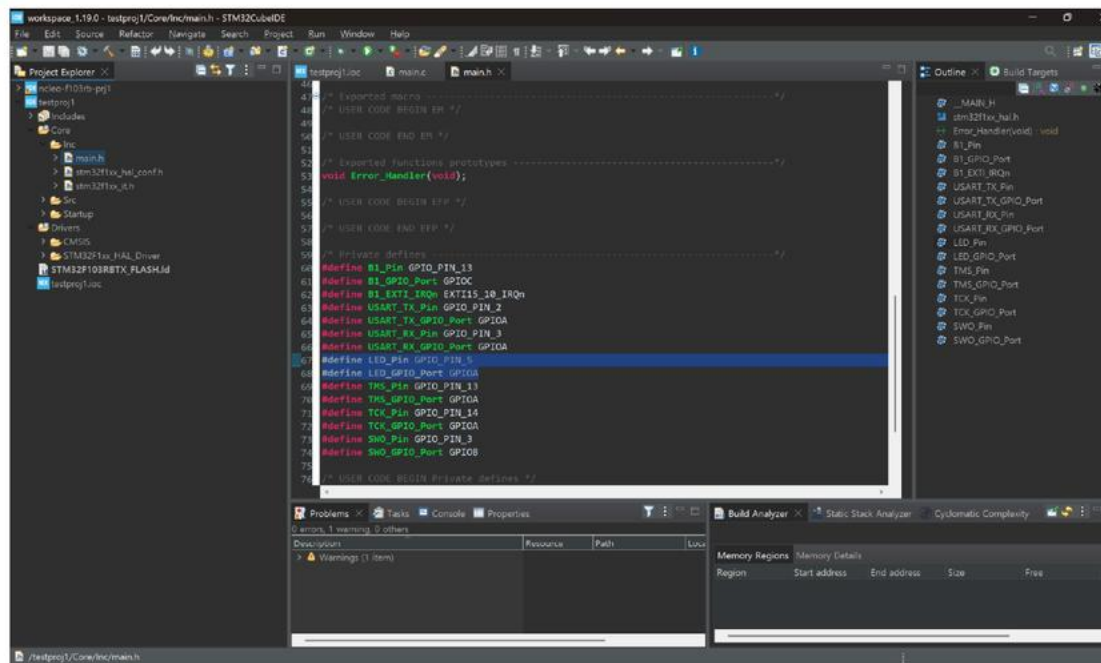
- ARM GPIO를 통한 LED 제어 실습
- 툴바 아이콘 중
“Device Configuration Tool Code Generation” 클릭



- ARM GPIO를 통한 LED 제어 실습
"perspective 를 open 하겠다"는 메시지가 뜬 후,
코드 창 열림



• ARM GPIO를 통한 LED 제어 실습



- ARM GPIO를 통한 LED 제어 실습
- Source Code for LED 점멸

```
/* USER CODE BEGIN WHILE */  
while (1)  
{  
/* USER CODE END WHILE */
```

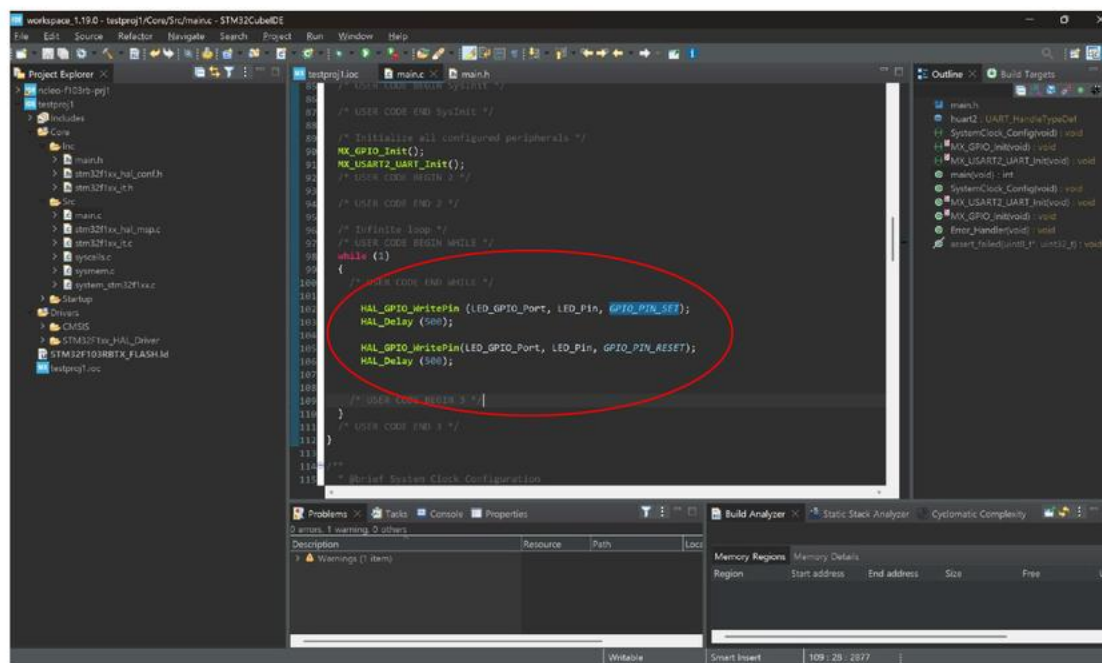
```
HAL_GPIO_WritePin (LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);  
HAL_Delay (500);
```

```
HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);  
HAL_Delay (500);
```

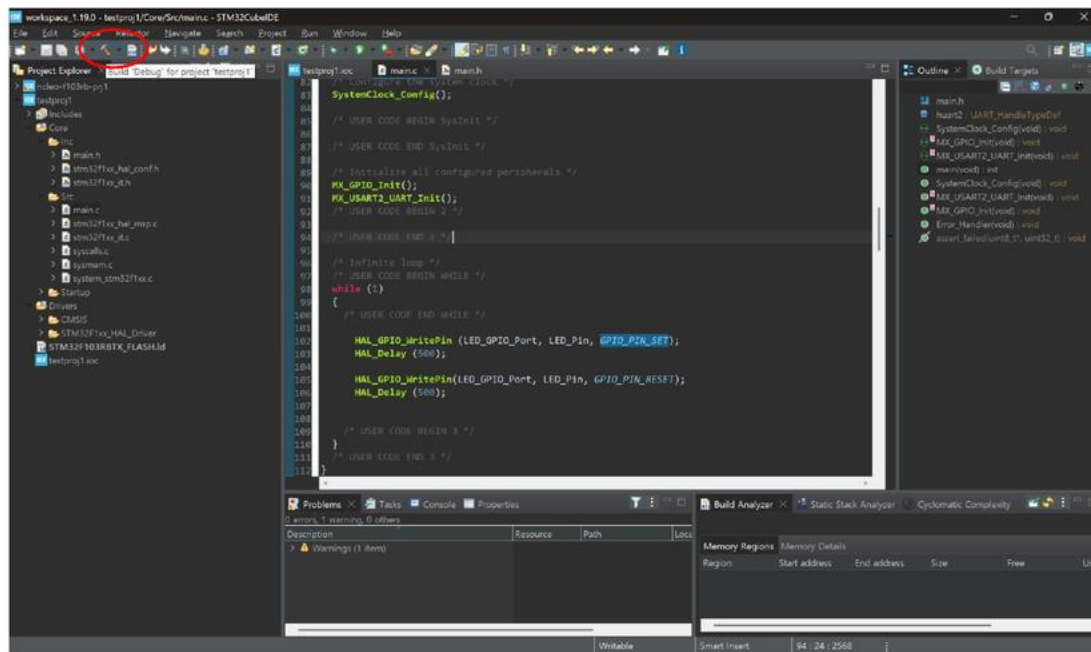
```
/* USER CODE BEGIN 3 */  
}  
/* USER CODE END 3 */
```

```
 *  
 * @param GPIOx: where x can be (A..G depending on device used) to select the GPIO peripheral  
 * @param GPIO_Pin: specifies the port bit to be written.  
 * This parameter can be one of GPIO_PIN_x where x can be (0..15).  
 * @param PinState: specifies the value to be written to the selected bit.  
 * This parameter can be one of the GPIO_PinState enum values:  
 * @arg GPIO_PIN_RESET: to clear the port pin  
 * @arg GPIO_PIN_SET: to set the port pin  
 * @retval None  
 */  
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
```

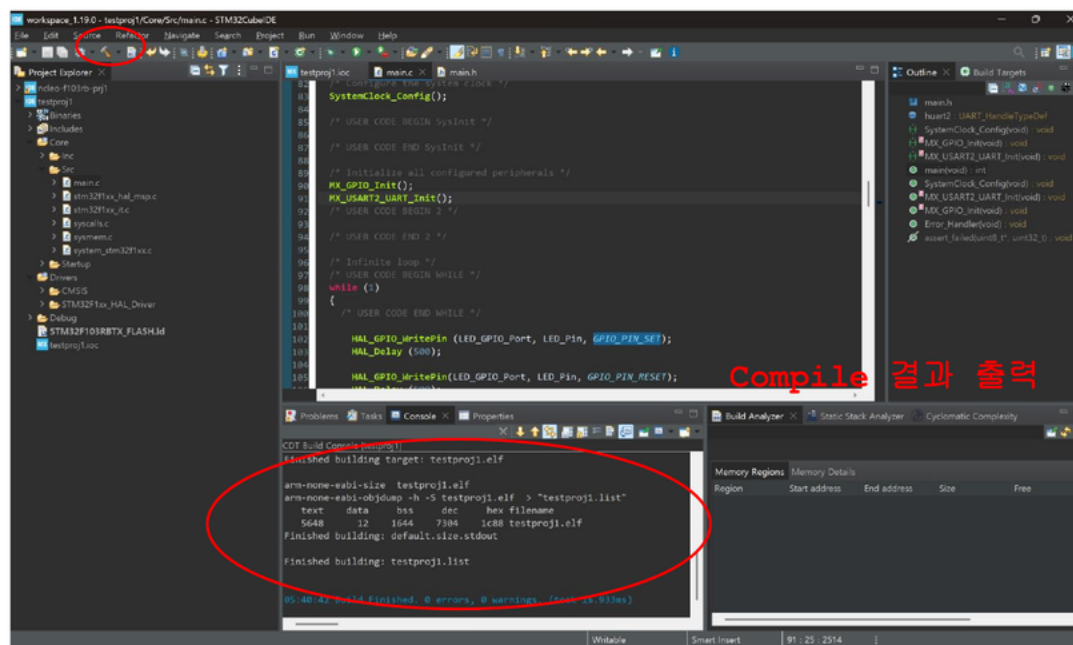
• ARM GPIO를 통한 LED 제어 실습



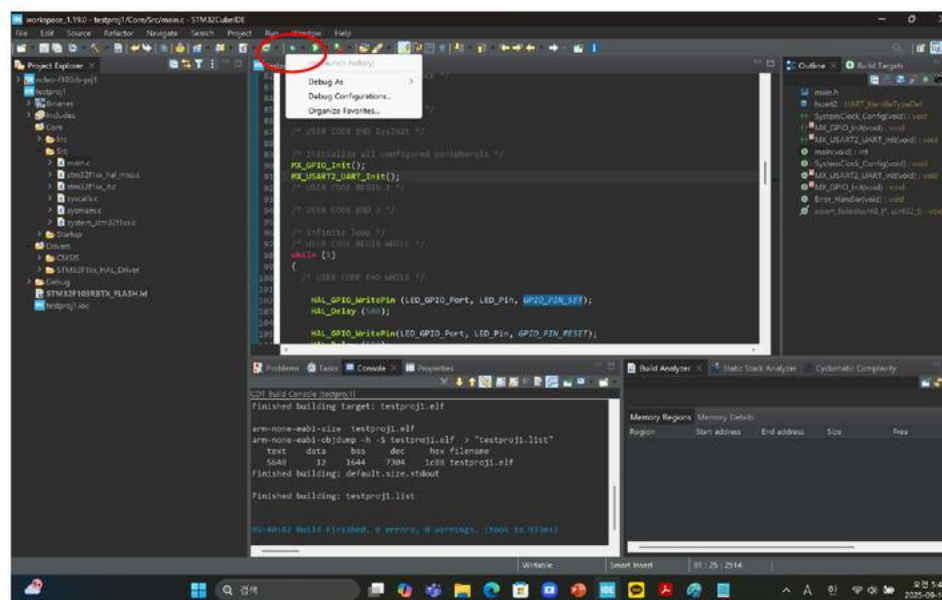
- ARM GPIO를 통한 LED 제어 실습
- 툴바 아이콘 중 “Build Debug for project 000” 버튼을 눌러 컴파일



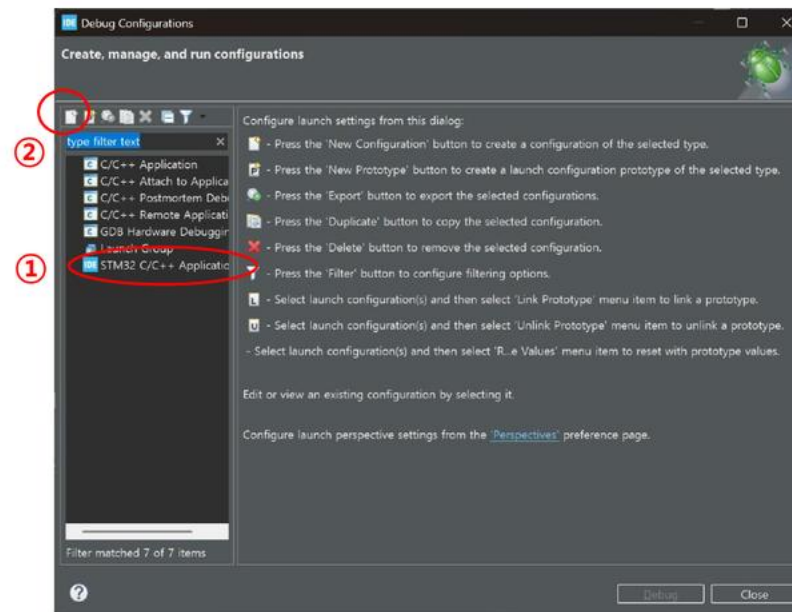
- ARM 프로세서 기술동향 이해
- 메인메뉴 아래 아이콘 중 “Build Debug for project 000” 버튼을 눌러 컴파일



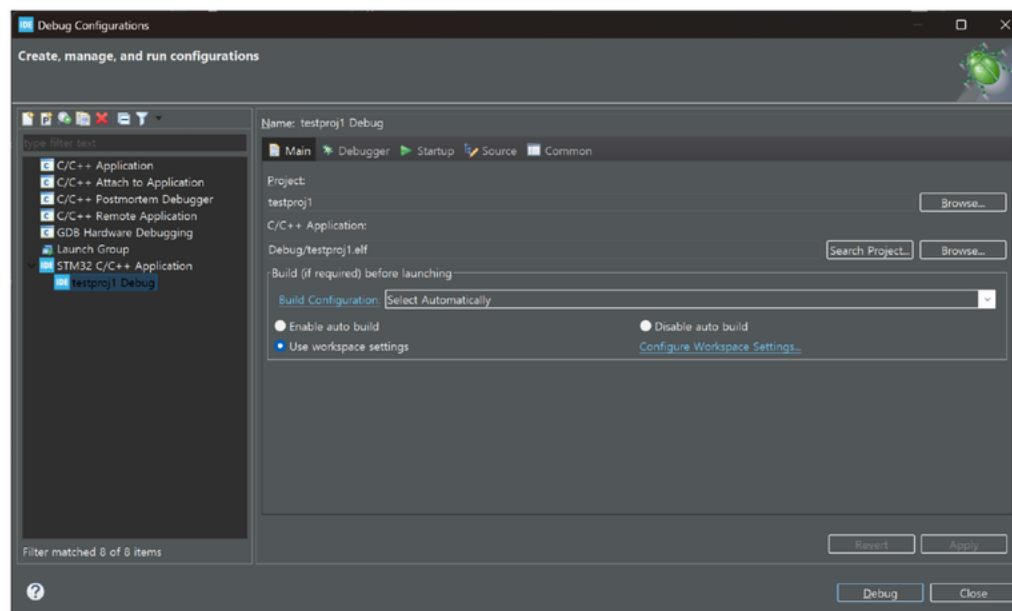
- ARM 프로세서 기술동향 이해
- 툴바에서 'Debug' 버튼의 “아래쪽 화살표” 클릭 후 'Debug Configuration' 실행



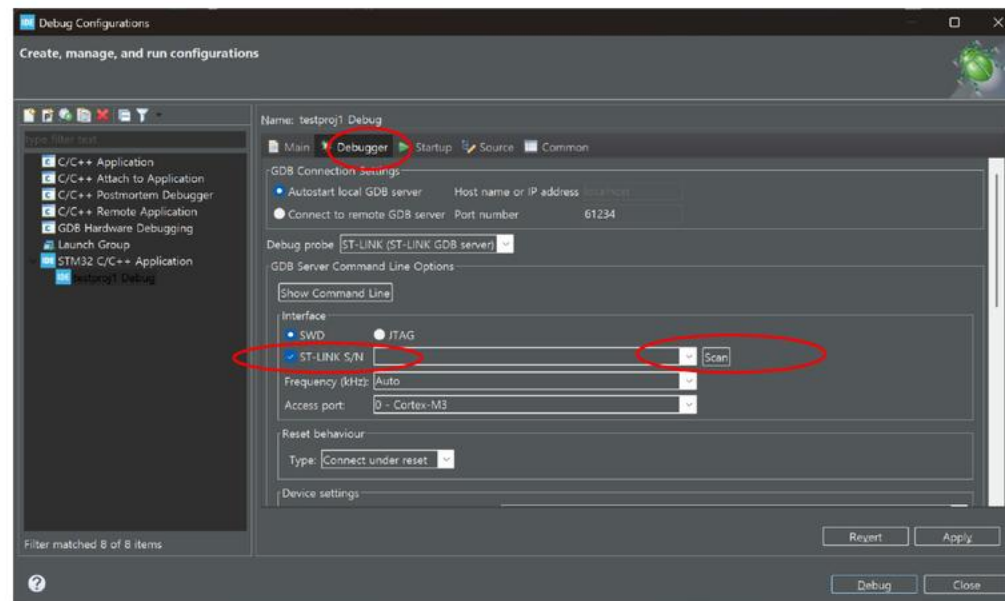
- ARM GPIO를 통한 LED 제어 실습
- 화면에서 “STM32 C/C++ Applications” 을 선택



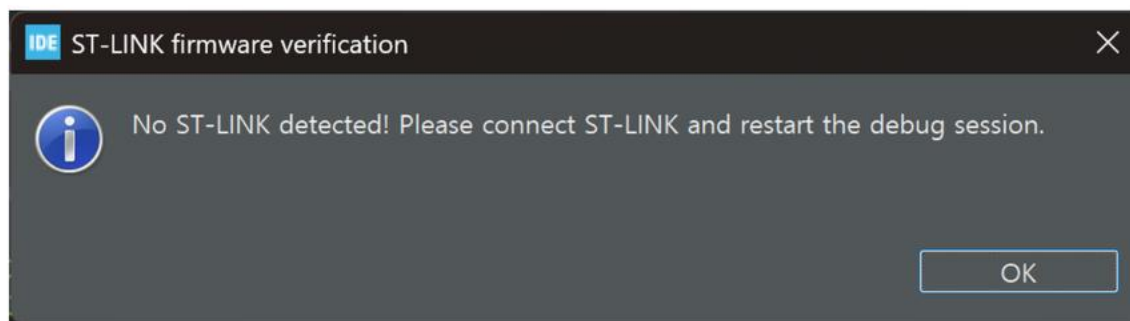
- ARM GPIO를 통한 LED 제어 실습
- 화면에서 “STM32 C/C++ Applications” 을 선택



- ARM GPIO를 통한 LED 제어 실습
- 화면에서 “STM32 C/C++ Applications” 을 선택



- ARM GPIO를 통한 LED 제어 실습
- 만약 아래 에러메시지 발생할 경우,
STM32 보드를 컴퓨터에 연결 필요
 - 이 과정은 컴파일된 프로그램을 STM32 보드에
다운로드하고 실행하는 과정이므로



SECTION 03

GPIO LED 제어 실습

STM32CubeIDE 프로젝트 생성부터 GPIO 설정, 코드 작성과 디버깅까지 단계별로 실습합니다.



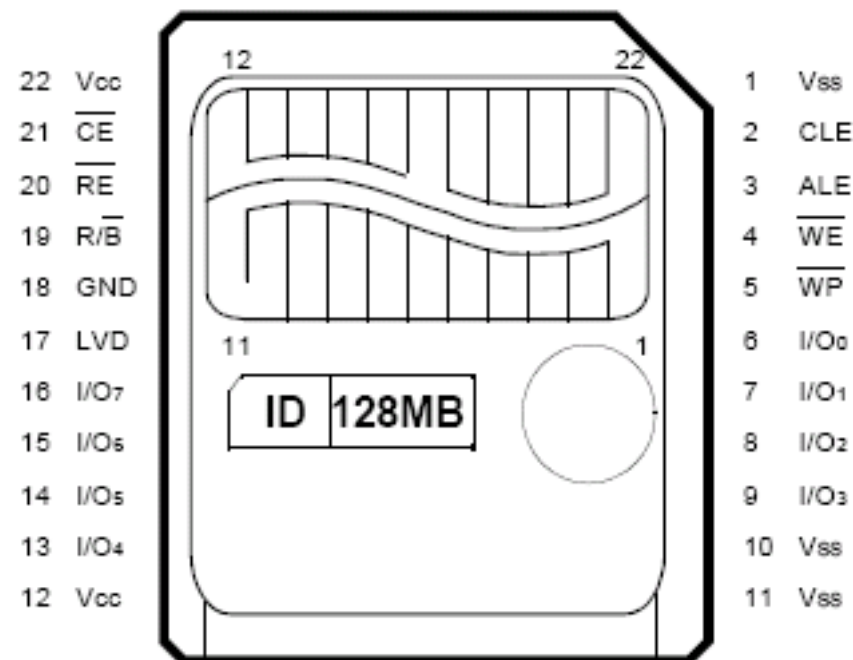
Flash 관련 Example 1)

DMA를 이용한 Flash->RAM 데이터 전송



- ✓ CPU가 직접 복사하지 않고 DMA가 메모리 간 데이터를 전송하는 원리를 설명할 수 있다.
- ✓ HAL_DMA_Init(), HAL_DMA_Start_IT(), HAL_DMA_RegisterCallback()의 역할을 구분할 수 있다.
- ✓ DMA 전송 완료/오류 인터럽트와 콜백의 실행 흐름을 설명할 수 있다.
- ✓ FLASH의 const 원본 버퍼와 SRAM의 목적지 버퍼를 디버거로 검증할 수 있다.
- ✓ DMA 설정의 증가 모드, 데이터 정렬, 우선순위가 왜 필요한지 설명할 수 있다.

SmartMedia™ CARD(SSFDC)



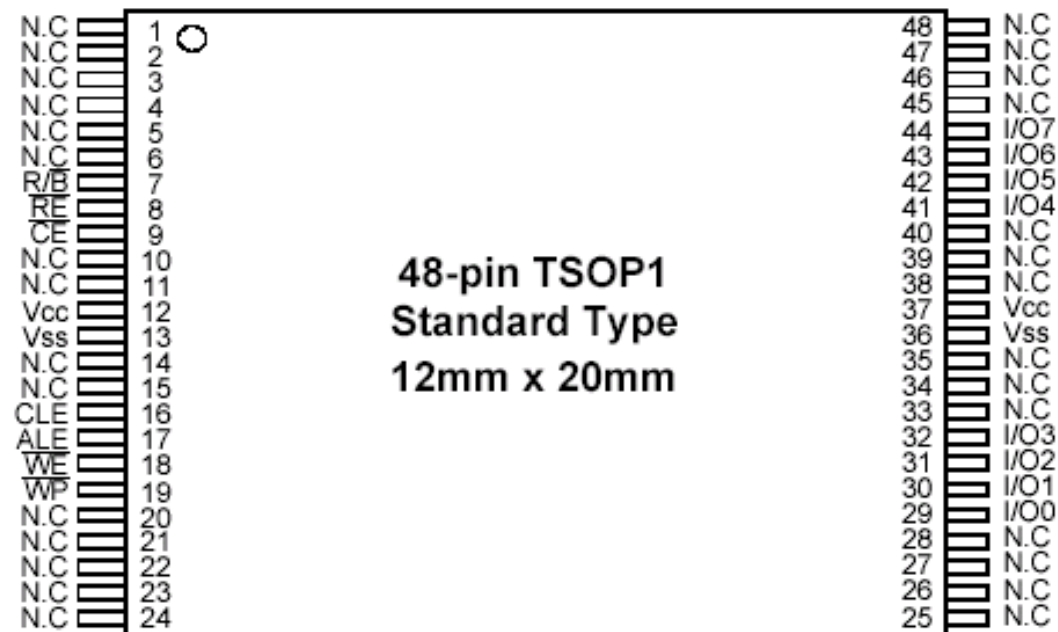
22 PAD SmartMedia™

PIN DESCRIPTION

Pin Name	Pin Function
I/O0 ~ I/O7	Data Input/Outputs
CLE	Command Latch Enable
ALE	Address Latch Enable
CE	Chip Enable
RE	Read Enable
WE	Write Enable
WP	Write Protect
LVD	Low Voltage Detect
GND	Ground
R/B	Ready/Busy output
Vcc	Power
Vss	Ground
N.C	No Connection

K9F1208UOM

PIN CONFIGURATION



PIN DESCRIPTION

Pin Name	Pin Function
I/O ₀ ~ I/O ₇	Data Input/Outputs
CLE	Command Latch Enable
ALE	Address Latch Enable
$\overline{\text{CE}}$	Chip Enable
$\overline{\text{RE}}$	Read Enable
$\overline{\text{WE}}$	Write Enable
$\overline{\text{WP}}$	Write Protect
$\overline{\text{R/B}}$	Ready/Busy output
Vcc	Power(+2.7V~3.6V)
Vss	Ground
N.C	No Connection

플래쉬 메모리 특징

NAND 플래시 메모리 명령

모든 NAND 플래시 동작은 명령 사이클로 시작
6개의 주요 제어 신호로 구성

I00 ~ I07 : 데이터 버스

CLE : High가 되면 현재 버스의 데이터는 명령임을 나타낸다.

ALE : High가 되면 현재 버스의 데이터는 주소임을 나타낸다.

nCE : Chip Select

nRE : 버스 읽기 신호

nWE : 버스 쓰기 신호

**6개의 제어 신호의 조합으로 이루어진 명령 사이클을 통해 여러 동작
(명령어 세트) 제어**

플래시 메모리 특징

NAND 플래시 메모리의 특징

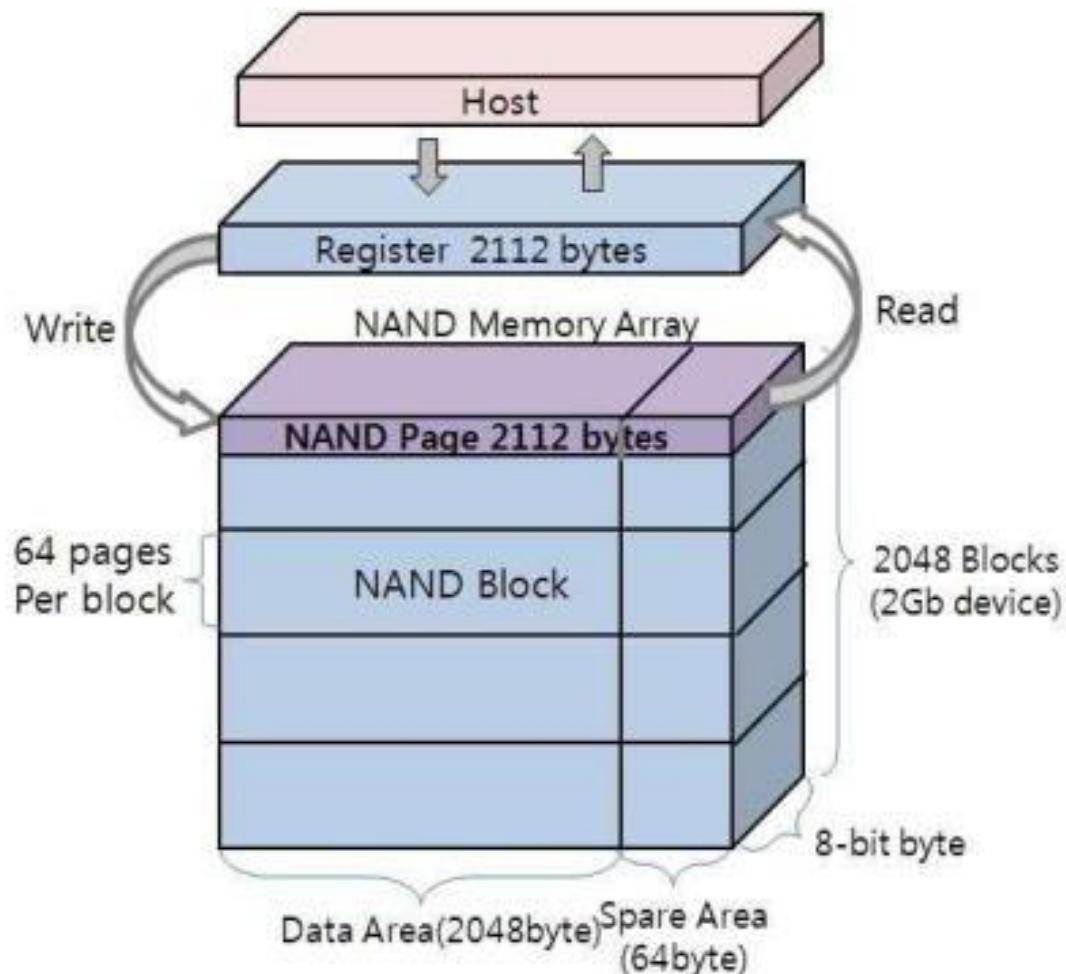
일정한 크기의 페이지(page) 단위로 읽기, 쓰기 연산 수행

블록 단위로 소거(erase) 연산 수행

덮어쓰기(overwrite) 연산이 불가능, 쓰기 전 소거(erase-before-write) 연산 발생하는 특징
읽기, 쓰기, 소거 연산 시 속도 차이가 있음

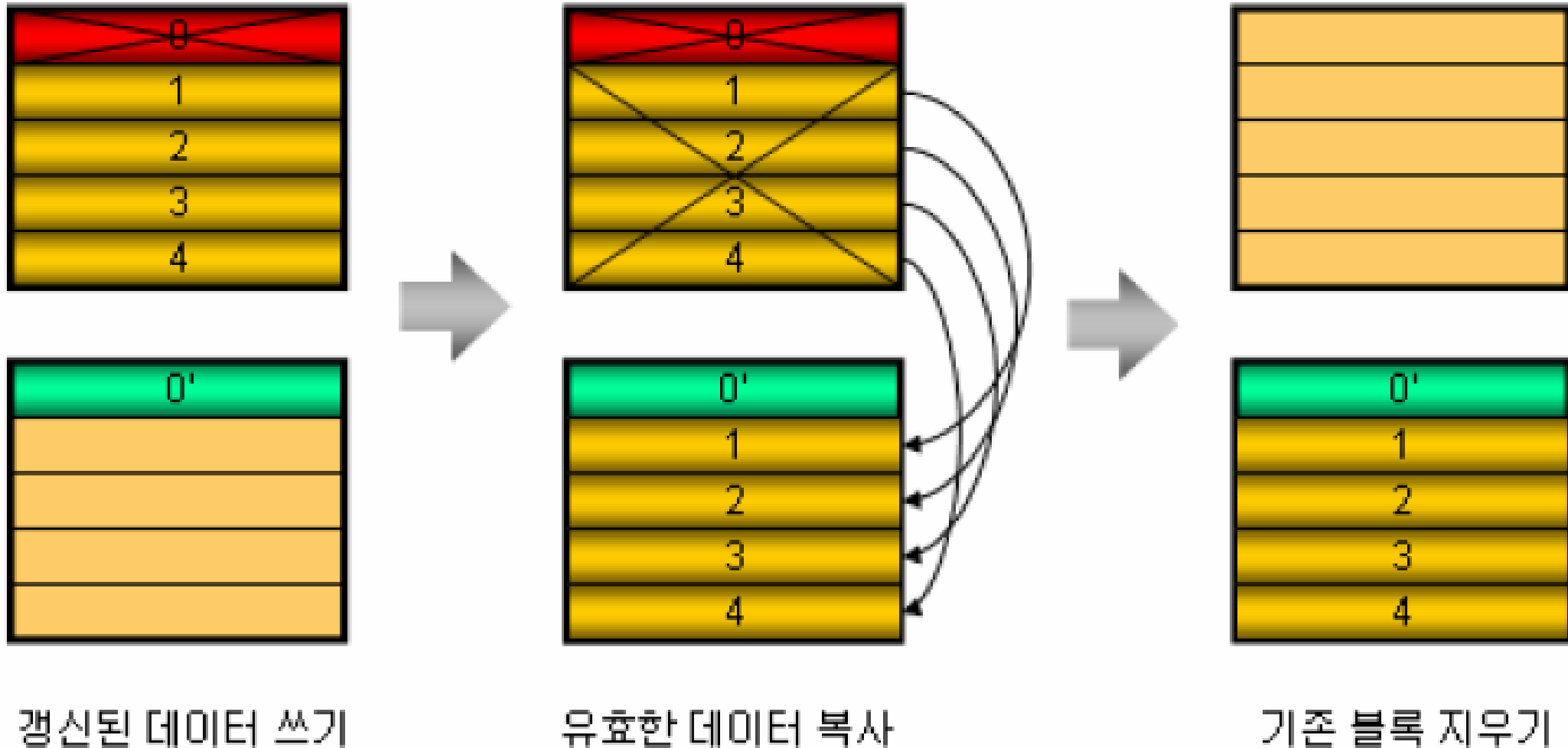
FTL(Flash Translation Layer)

파일 시스템에서 플래시 메모리를 논리적 주소로 바라볼 수 있게 해주는 매개체 역할



구조 및 특징

플래시 메모리는 덮어쓰기(**overwrite**)가 되지 않음



구조 및 특징

플래시메모리는 덮어쓰기(**overwrite**) 동작이 불가함.

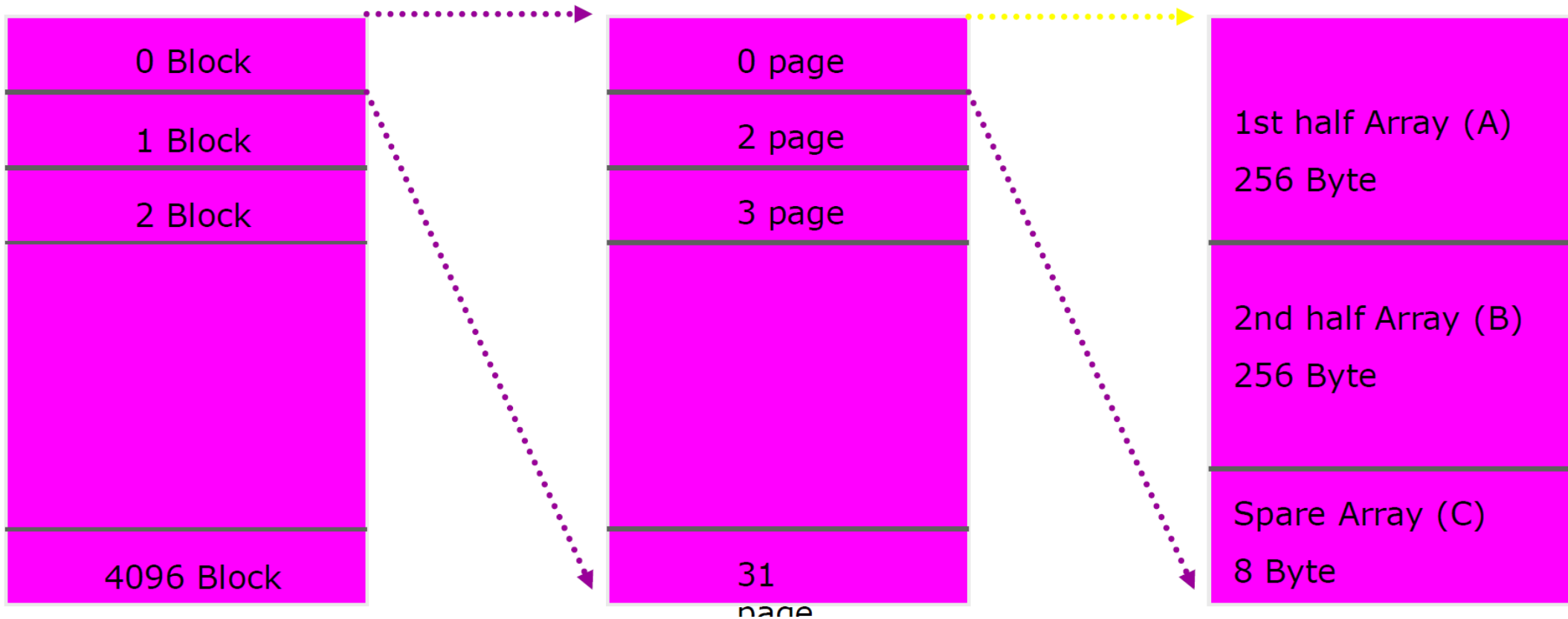
	Access Time		
	Read	Write	Erase
NOR	$23\mu s$	$28ms$	$1.2sec$
NAND	$156\mu s$	$652\mu s$	$2ms$

각 다른 Flash 구조의 내용

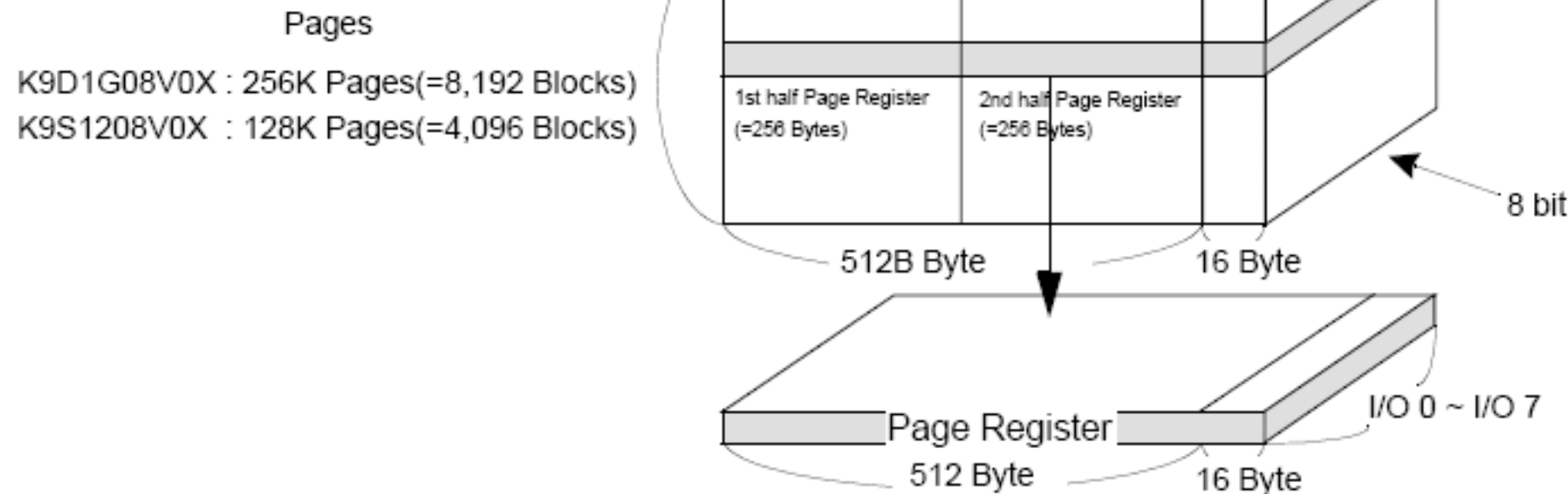
Nand Flash 메모리의 주소를 대입하여야 데이터를 읽고 쓸 수 있다

외부와 연결된 I/O 버스는 Flash 메모리 블록과 직접 연결되어 있지 않고 버퍼를 통해서 읽고 쓴다

페이지 단위로 읽고 쓸 수 있는 구조로 구성



플래쉬 메모리 구조



ARRAY ORGANIZATION

	1 Page	1 Block	1 Device
K9D1G08V0X	528 Byte	528 Byte x 32 Pages	528Byte x 32Pages x 8,192 Blocks
K9S1208V0X	528 Byte	528 Byte x 32 Pages	528Byte x 32Pages x 4,096 Blocks

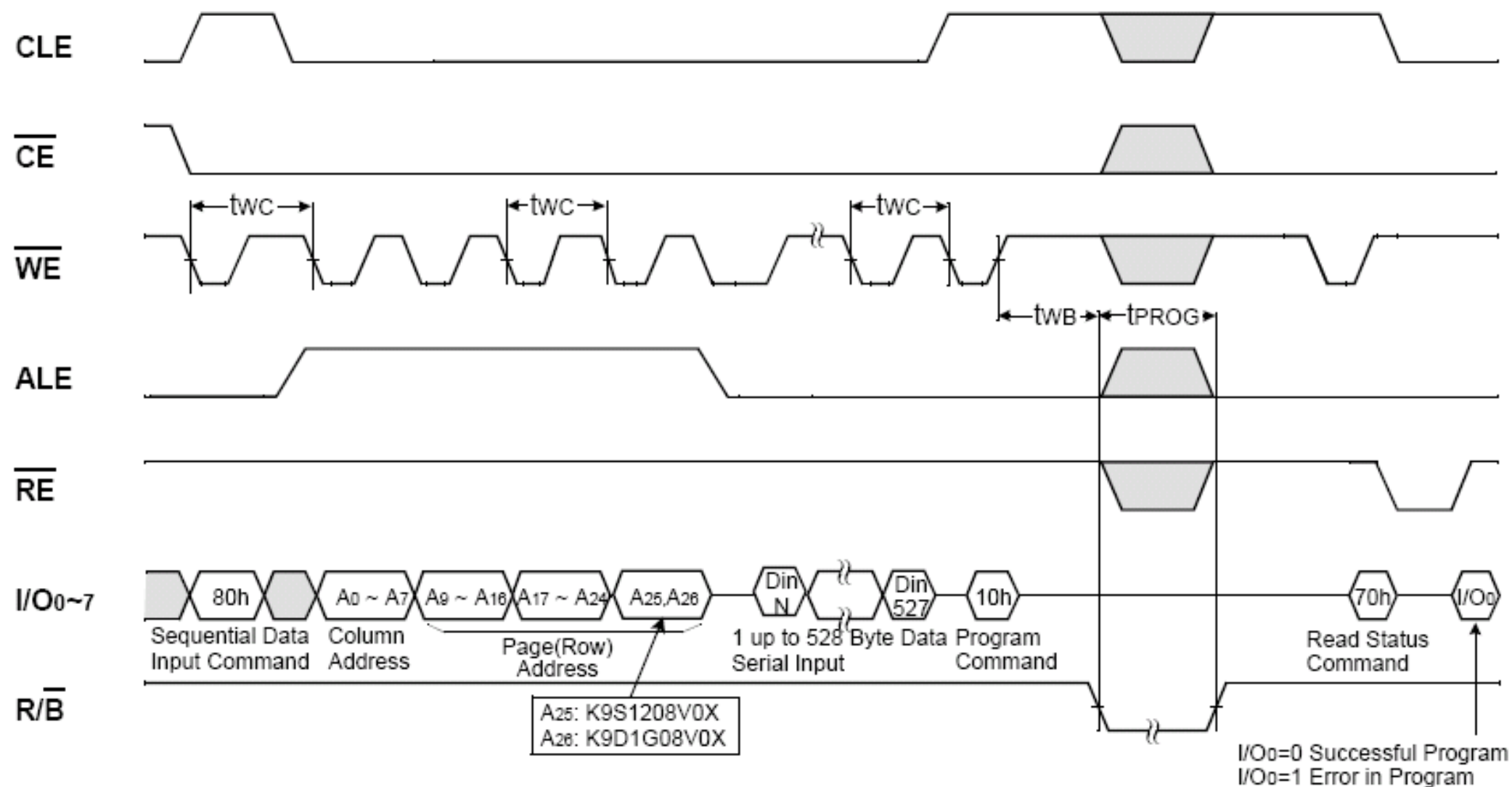
각 cycle별 I/O 포트의 내용

	I/O 0	I/O 1	I/O 2	I/O 3	I/O 4	I/O 5	I/O 6	I/O 7	Column Address Row Address (Page Address)
1st Cycle	A0	A1	A2	A3	A4	A5	A6	A7	
2nd Cycle	A9	A10	A11	A12	A13	A14	A15	A16	
3rd Cycle	A17	A18	A19	A20	A21	A22	A23	A24	
4th Cycle	A25	A26	*L	*L	*L	*L	*L	*L	

명령어 세트

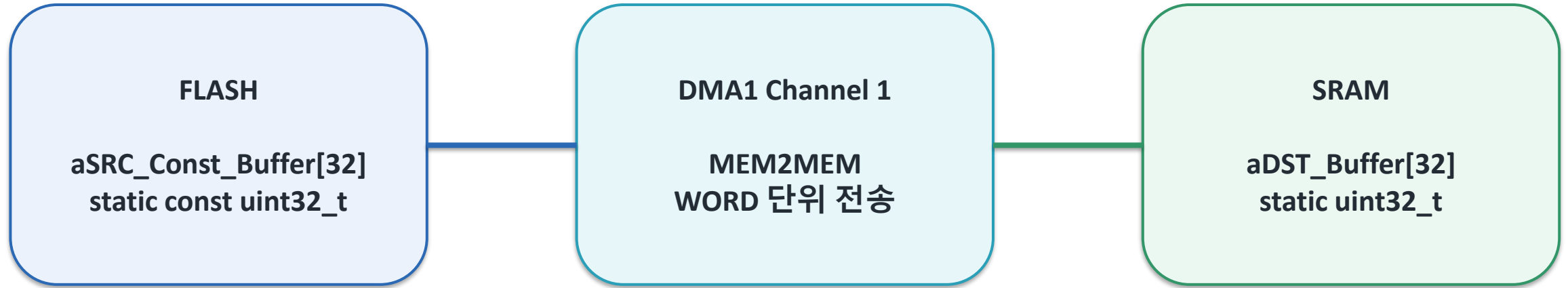
Function	1st. Cycle	2nd. Cycle	3rd. Cycle	Acceptable Command during
Read 1	00h/01h ⁽¹⁾	-	-	
Read 2	50h	-	-	
Read ID	90h	-	-	
Reset	FFh	-	-	O
Page Program (True)	80h	10h	-	
Page Program (Dummy)	80h	11h	-	
Page Program (Multi Block Program)	80h	15h	-	
Block Erase	60h	D0h	-	
Multi-Plane Block Erase	60h---60h	D0h	-	
Read Status	70h	-	-	O
Read Multi-Plane Status	71h ⁽²⁾	-	-	O

페이지 프로그램 동작



예제의 데이터 이동 구조

32-bit word 32개를 FLASH에서 SRAM으로 복사.



원본은 `const`이므로 일반적으로
읽기 전용 FLASH 영역에 배치

CPU 대신 DMA가
버스 마스터로 복사 수행

전송 후 원본과 동일한
32개 word가 저장

총 전송량 = 32 words × 4 bytes = 128 bytes

10.2 DMA main features

- Single AHB master
- Peripheral-to-memory, memory-to-peripheral, memory-to-memory and peripheral-to-peripheral data transfers
- Access, as source and destination, to on-chip memory-mapped devices such as flash memory, SRAM, and AHB and APB peripherals
- All DMA channels independently configurable:
 - Each channel is associated either with a DMA request signal coming from a peripheral, or with a software trigger in memory-to-memory transfers. This configuration is done by software.
 - Priority between the requests is programmable by software (four levels per channel: very high, high, medium, low) and by hardware in case of equality (such as request to channel 1 has priority over request to channel 2).
 - Transfer size of source and destination are independent (byte, half-word, word), emulating packing and unpacking. Source and destination addresses must be aligned on the data size.
 - Support of transfers from/to peripherals to/from memory with circular buffer management
 - Programmable number of data to be transferred: 0 to $2^{16} - 1$
- Generation of an interrupt request per channel. Each interrupt request is caused from any of the three DMA events: transfer complete, half transfer, or transfer error.

초기화 단계

main() 진입 후 DMA 전송을 시작하기 전까지의 설정 순서입니다.



MX_DMA_Init() 설정

Memory-to-Memory 전송에 필요한 DMA Channel 1 파라미터

Request	DMA_REQUEST_MEM2MEM	메모리 간 전송 요청
Direction	DMA_MEMORY_TO_MEMORY	Source → Destination
PeriphInc	DMA_PINC_ENABLE	Source 주소를 word마다 증가
MemInc	DMA_MINC_ENABLE	Destination 주소를 word마다 증가
Data Alignment	WORD / WORD	32-bit 단위로 읽고 씀
Mode	DMA_NORMAL	BUFFER_SIZE만큼 전송 후 종료
Priority	DMA_PRIORITY_LOW	DMA 채널 우선순위 낮음
NVIC	DMA1_Channel1_IRQn	전송 완료/오류 인터럽트 허용

```
216 static void MX_DMA_Init(void)
217 {
218
219     /* DMA controller clock enable */
220     __HAL_RCC_DMA1_CLK_ENABLE();
221
222     /* Configure DMA request hdma_memtomem_dma1_channel1 on DMA1_Channel1 */
223     hdma_memtomem_dma1_channel1.Instance = DMA1_Channel1;
224     hdma_memtomem_dma1_channel1.Init.Request = DMA_REQUEST_MEM2MEM;
225     hdma_memtomem_dma1_channel1.Init.Direction = DMA_MEMORY_TO_MEMORY;
226     hdma_memtomem_dma1_channel1.Init.PeriphInc = DMA_PINC_ENABLE;
227     hdma_memtomem_dma1_channel1.Init.MemInc = DMA_MINC_ENABLE;
228     hdma_memtomem_dma1_channel1.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;
229     hdma_memtomem_dma1_channel1.Init.MemDataAlignment = DMA_MDATAALIGN_WORD;
230     hdma_memtomem_dma1_channel1.Init.Mode = DMA_NORMAL;
231     hdma_memtomem_dma1_channel1.Init.Priority = DMA_PRIORITY_LOW;
232     if (HAL_DMA_Init(&hdma_memtomem_dma1_channel1) != HAL_OK)
233     {
234         Error_Handler( );
235     }
236 }
```

※ HAL의 MEM2MEM 구현에서는 'Periph' 필드명이 실제 주변장치를 뜻하지 않고 source-side 설정으로 사용되는 점에 주의

DMA 전송 시작

HAL_DMA_Start_IT()가 source, destination, length를 DMA에 설정하고 인터럽트 방식으로 시작합니다.

```
HAL_DMA_Start_IT(  
    &hdma_memtomem_dma1_channel1,  
    (uint32_t)&aSRC_Const_Buffer,  
    (uint32_t)&aDST_Buffer,  
    BUFFER_SIZE  
);
```

Source Address

FLASH의 aSRC_Const_Buffer

Destination

SRAM의 aDST_Buffer

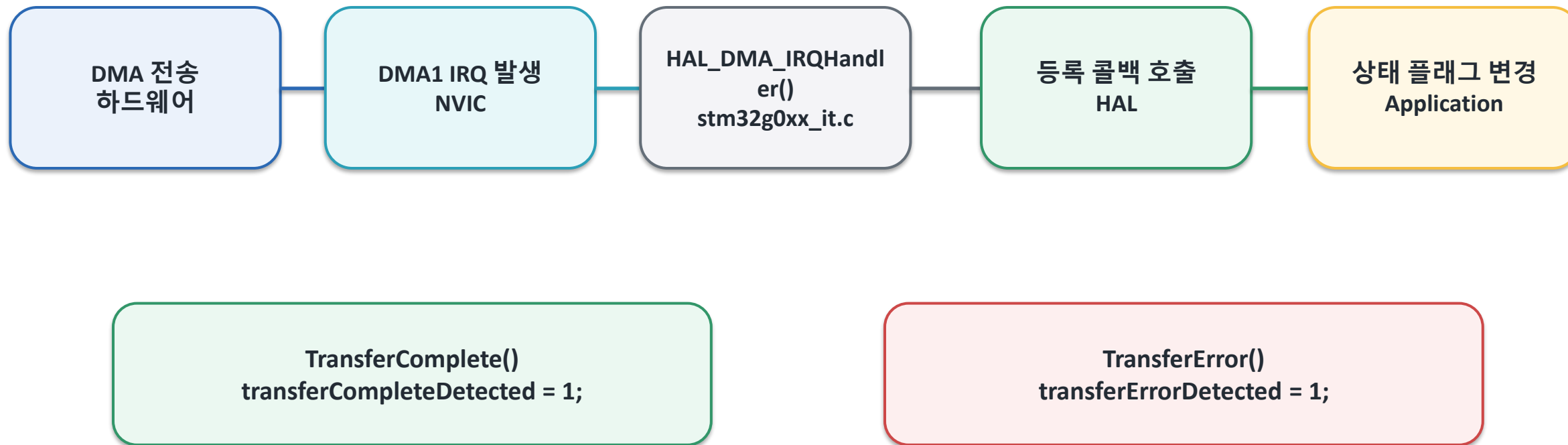
Length

BUFFER_SIZE = 32 words

HAL_OK → DMA가 백그라운드에서 전송 진행
HAL_OK가 아니면 Error_Handler()로 진입

인터럽트와 콜백 동작

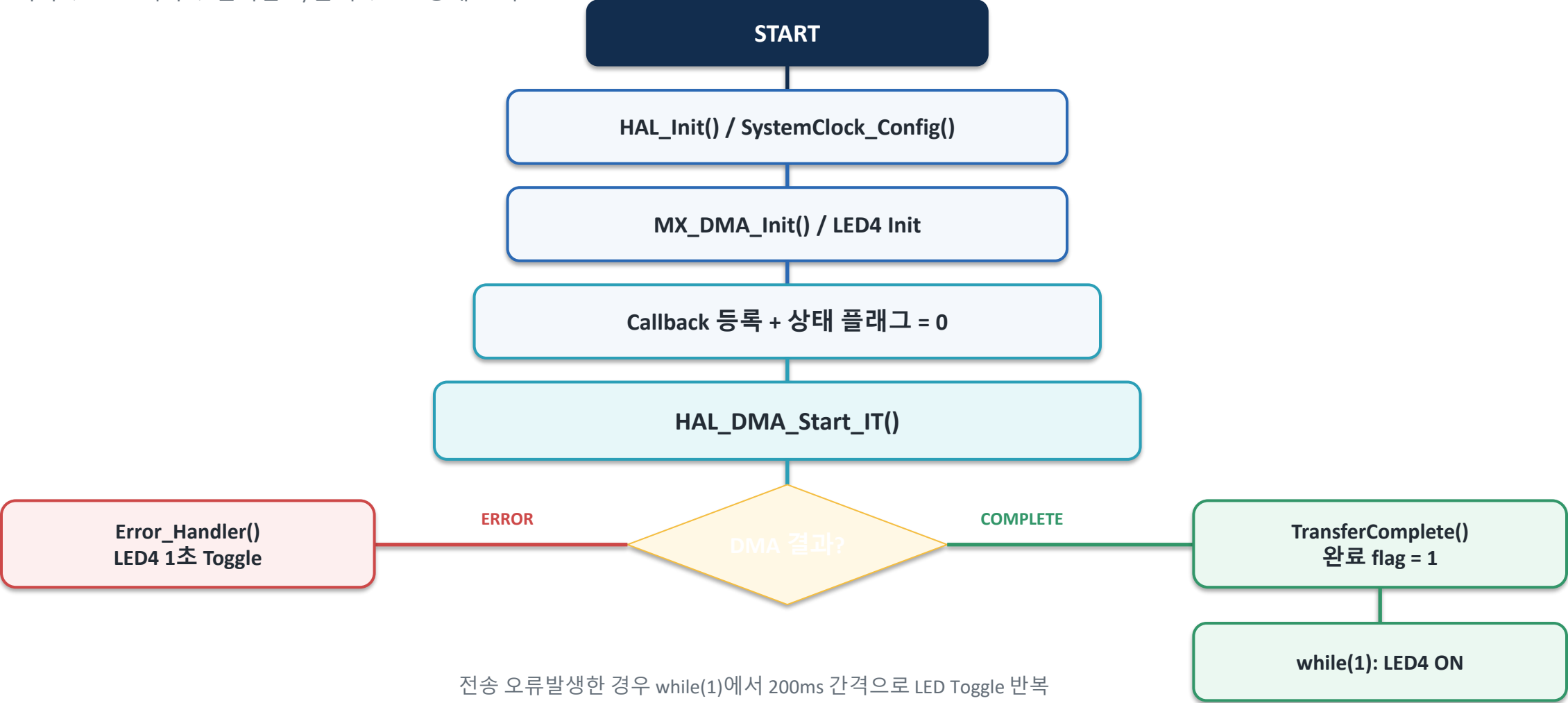
DMA 완료/오류 플래그는 ISR을 거쳐 등록된 콜백에서 갱신됩니다.



중요: main.c 밖의 IRQ Handler에서 HAL_DMA_IRQHandler(&hdma_memtomem_dma1_channel1)를 호출해야 HAL 콜백이 실행됩니다.

전체 프로그램 Flow Chart

초기화 → DMA 시작 → 인터럽트/콜백 → LED 상태 표시



주요 함수 정리

HAL_Init()

HAL, Flash interface, SysTick 등 기본 초기화

SystemClock_Config()

HSI와 PLL을 설정하여 시스템/버스 클록 구성

MX_DMA_Init()

DMA1 clock, Channel 1 파라미터, NVIC IRQ 활성화

HAL_DMA_RegisterCallback()

DMA 완료/오류 시 실행할 사용자 콜백 연결

HAL_DMA_Start_IT()

DMA 주소/길이 설정 후 인터럽트 모드로 전송 시작

TransferComplete()

완료 플래그를 1로 설정

TransferError()

오류 플래그를 1로 설정

Error_Handler()

치명적 오류 시 LED4를 1초 주기로 토글하며 정지

```
/* Initialize LED4 */
BSP_LED_Init(LED4);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
/* Reset transferErrorDetected to 0, it will be set to 1 if a transfer error is detected */
transferErrorDetected = 0;
/* Reset transferCompleteDetected to 0, it will be set to 1 if a transfer is correctly completed */
transferCompleteDetected = 0;

/* Select Callbacks functions called after Transfer complete and Transfer error */
HAL_DMA_RegisterCallback(&hdma_memtomem_dma1_channel1, HAL_DMA_XFER_CPLT_CB_ID, TransferComplete);
HAL_DMA_RegisterCallback(&hdma_memtomem_dma1_channel1, HAL_DMA_XFER_ERROR_CB_ID, TransferError);

/* Configure the source, destination and buffer size DMA fields and Start DMA Channel/Stream transfer */
/* Enable All the DMA interrupts */
if (HAL_DMA_Start_IT(&hdma_memtomem_dma1_channel1, (uint32_t)&aSRC_Const_Buffer, (uint32_t)&aDST_Buffer, BUFFER_SIZE) != HAL_OK)
{
    /* Transfer Error */
    Error_Handler();
}

/* Infinite loop */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    if (transferErrorDetected == 1)
    {
        /* Toggle LED4 with a period of 200 ms */
        BSP_LED_Toggle(LED4);
        HAL_Delay(200);
    }

    if (transferCompleteDetected == 1)
    {
        /* Turn LED4 on*/
        BSP_LED_On(LED4);
        transferCompleteDetected = 0;
    }
}
```

실습 및 디버깅 포인트

DMA가 실제로 데이터를 복사했는지 확인하는 방법

전송 전 aDST_Buffer를 Watch/Memory 창에서 확인한다.

HAL_DMA_Start_IT() 실행 후 DMA1 Channel 1 레지스터를 관찰한다.

TransferComplete()에 breakpoint를 걸어 완료 인터럽트를 확인한다.

전송 후 aSRC_Const_Buffer와 aDST_Buffer의 32개 word가 동일한지 비교한다.

오류 실습 시 주소/정렬/IRQ 설정을 점검하고 transferErrorDetected 변화를 확인한다.

검증 조건

`src[i] == dst[i]`
(i = 0 ... 31)

정상 완료

`transferCompleteDetected = 1`
LED4 ON

전송 오류

`transferErrorDetected = 1`
LED4 200 ms Toggle

권장: memcpy() 방식과 CPU 점유율/동작 구조 비교

코드 분석 시 주의사항

BUFFER_SIZE는 바이트 수가 아니라 DMA 전송 단위(word)의 개수임(이 예제에서는 $32 \times 4 = 128$ bytes)

MEM2MEM에서도 HAL 구조체 필드명이 PeriphInc / PeriphDataAlignment로 남아 있으므로 실제 주변장치 전송으로 오해 금지.

콜백 함수는 자동으로 호출되는 것이 아니라 DMA IRQ → HAL_DMA_IRQHandler() 경로가 구성되어 있어야 함

transferCompleteDetected와 transferErrorDetected는 ISR/콜백과 main loop가 함께 접근하므로 __IO(volatile 성격)로 선언 된다는 점.

완료 후 transferCompleteDetected를 0으로 되돌리므로 LED ON 처리는 한 번만 수행됨.

① DMA 초기화
Clock · Channel · NVIC

② 콜백 등록
Complete · Error

③ 전송 시작
Source · Destination ·
Length

④ IRQ 처리
Flag 갱신 · LED 상태 표시

DMA는 '복사를 대신 수행하는 하드웨어 엔진'이며, CPU는 설정 → 다른 작업 → 완료 통지 처리의 구조로 동작한다.

Flash 관련 Example 2)

STM32G0 내장 플래쉬 메모리(NOR Flash)에서 Flash Erase / Program 동작 예제

- ✓ CPU가 직접 복사하지 않고 DMA가

예제에서 수행하는 내용

예제는 STM32G0xx 내장 Flash를 지우고 다시 프로그램하는 절차를 보여줌

대상 영역
FLASH_USER_START_ADDR
= Page 16

FLASH_USER_END_ADDR
= Page 127 끝

기록 데이터
DATA_64
0x1234567812345678

검증 데이터
DATA_32
0x12345678

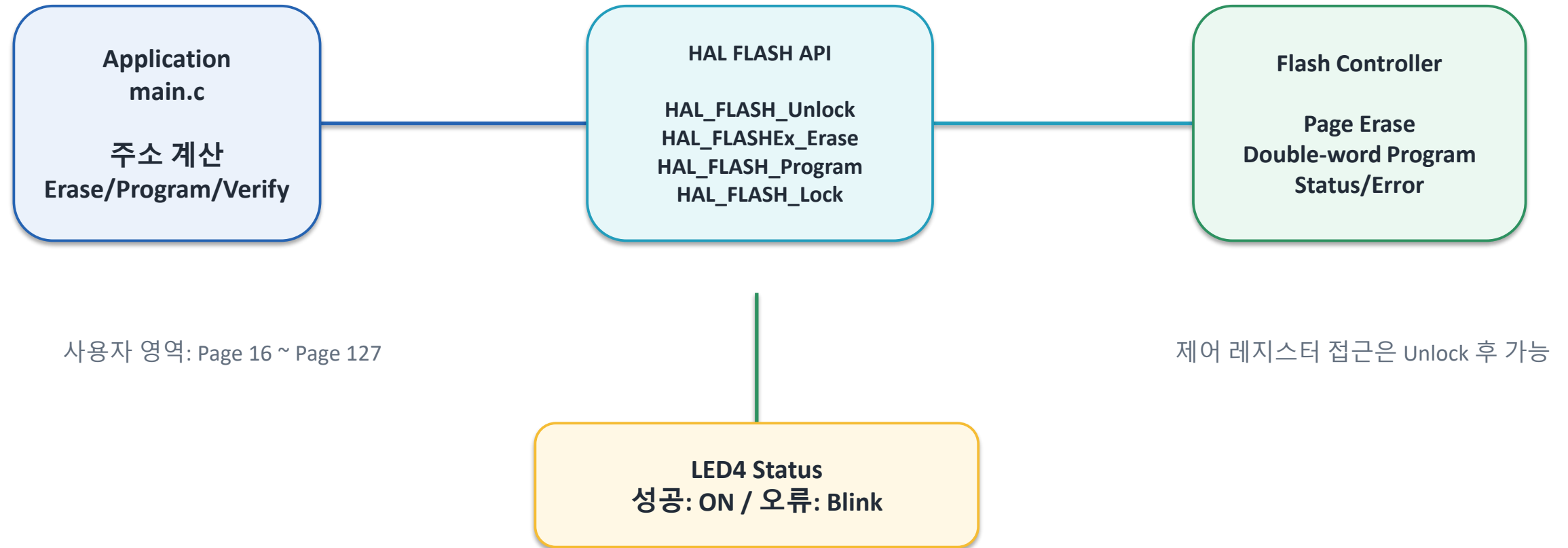
상태 표시
LED4 ON = 성공
빠른/느린 Blink = 오류

코드는 Page 단위 Erase 후, 64-bit double-word 단위로 Program하고, 32-bit 단위로 Read-back 검증을 수행합니다.

강의 포인트: Flash는 RAM처럼 덮어쓰는 메모리가 아니며, 일반적으로 먼저 Erase하여 1 상태로 만든 뒤 Program으로 필요한 bit를 0으로 바꿉니다.

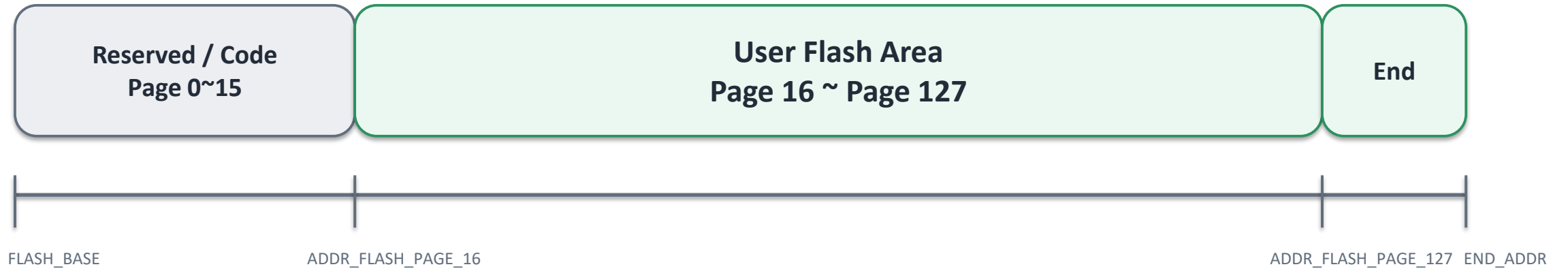
예제 수행 구조

Application 코드가 HAL을 통해 Flash Controller를 제어하고, 결과는 LED4로 표시됩니다.



Flash 사용자 영역 설정

예제는 전체 Flash 중 Page 16부터 Page 127까지를 사용자 Program 영역으로 사용.



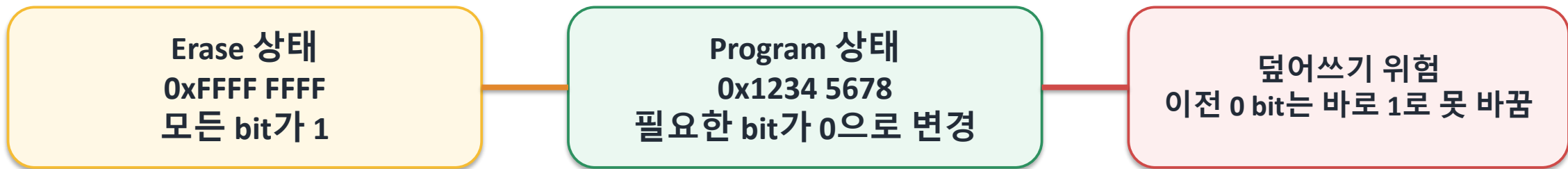
FirstPage = GetPage(START)

NbOfPages = EndPage - FirstPage + 1

BankNumber = GetBank(START)

왜 Erase 후 Program 하는가?

Flash cell은 Program으로 1→0 방향 변경은 가능하지만, 0→1 복구는 Page Erase가 필요합니다.



Erase 단위는 page 이며, Program 단위는 이 예제에서 Double Word(64-bit)입니다.
따라서 쓰려는 영역 전체를 먼저 Erase하고, 정해진 Program 단위와 정렬을 맞춰 기록합니다.

정확한 표현: STM32G0 내장 Flash는 일반 NAND Flash가 아니라 embedded Flash이며, Erase-before-Program 원칙을 따릅니다.

1. 용량 및 बैं크(Bank) 구조

- **메모리 용량 범위:** 하위 라인업의 **16KB**부터 고성능 제품군의 최대 **512KB**까지 폭넓은 용량 제공
- **듀얼 बैं크(Dual Bank) 지원:** 대용량 제품군(예: STM32G0B1 등 256KB~512KB 모델)은 플래시가 2개의 독립된 बैं크로 나뉩니다. 이를 통해 애플리케이션이 구동 중인 상태에서 다른 बैं크의 펌웨어를 다운로드하거나 업데이트할 수 있는 **RWW(Read-While-Write)** 기능을 지원하여 실시간 FOTA(Firmware Over-The-Air) 구현에 유리
- **페이지(Page) 구조:** 지우기(Erase) 및 쓰기(Program)의 최소 단위인 페이지 크기는 모델에 따라 다르나 일반적으로 **2KB** 단위를 사용

2. 고속 읽기 최적화 및 캐시

- **64MHz 제로 웨이트 스테이트(Zero-wait state):** 기본적으로 플래시 메모리는 CPU 속도보다 느리지만, STM32G0는 내부 Prefetch 버퍼와 Instruction 캐시(**128비트 캐시 라인**)를 내장하고 있어 64MHz 최대 주파수 동작 시에도 대기 시간(Wait state)으로 인한 성능 저하를 최소화

3. 신뢰성 및 수명 특징

- **ECC (Error Correction Code) 내장:** 하드웨어 수준에서 플래시 메모리 데이터의 오류를 스스로 감지하고 수정하는 내장 ECC 기능이 포함되어 있어 가전 및 산업용 시스템에서 요구되는 높은 신뢰성을 보장
- **지우기/쓰기 수명(Endurance):** 일반적인 실온 기준 최소 **10,000회(10K cycles)** 이상의 프로그램/삭제 사이클을 보장

main() 함수 내 실행 순서

초기화부터 Flash 잠금, 검증, LED 상태 표시까지 한 번 실행한 후 무한 루프에 진입합니다.

1	HAL_Init()	기본 HAL/SysTick 초기화
2	SystemClock_Config()	64 MHz 시스템 클록 구성
3	BSP_LED_Init(LED4)	상태 LED 초기화
4	HAL_FLASH_Unlock()	Flash 제어 레지스터 접근 허용
5	Erase 설정/실행	Page 16~127 Erase
6	Program Loop	DATA_64를 8 byte씩 기록
7	HAL_FLASH_Lock()	Flash 제어 보호
8	Verify Loop	DATA_32로 4 byte씩 읽어 검증

FLASH_EraseInitStruct 설정 분석

Erase 대상과 범위는 구조체 필드로 HAL에 전달됩니다.

필드	설정값	의미
TypeErase	FLASH_TYPEERASE_PAGES	Page 단위 Erase
Banks	BankNumber	Erase할 Flash bank
Page	FirstPage	Erase 시작 page
NbPages	NbOfPages	Erase할 page 개수
PageError	출력 변수	오류 발생 page 확인용

오류 발생 시 PageError에는 실패한 page 정보가 저장되며, 상세 오류는 HAL_FLASH_GetError()로 확인할 수 있습니다.

주요 함수와 역할

HAL_FLASH_Unlock()

Flash control register 접근 잠금 해제

__HAL_FLASH_CLEAR_FLAG()

OPTVERR 등 상태 플래그 초기화

GetPage(Address)

주소가 속한 Flash page 번호 계산

GetBank(Address)

주소가 속한 Flash bank 선택

HAL_FLASHEx_Erase()

EraseInitStruct 조건에 따라 page erase 실행

HAL_FLASH_Program()

지정 주소에 double-word 데이터 기록

HAL_FLASH_Lock()

의도치 않은 Flash 제어 방지를 위해 다시 잠금

Error_Handler()

시스템 오류 발생 시 무한 루프 진입

Read-back Verify Flow Chart

기록 후 32-bit 단위로 읽어 DATA_32와 비교합니다.

